

A Minor Project  
on  
**Predicting Home Prices with Regression Analysis Using the  
Boston Housing Dataset**  
**A PROJECT THESIS**  
*submitted*  
*in the partial fulfillment of the requirements for  
the award of the degree of*  
**Bachelor of Technology**  
in  
**COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**

By

|                   |   |            |
|-------------------|---|------------|
| Md. Ammaar Quadri | - | 21E41A3312 |
| M. Sahithi        | - | 21E41A3313 |
| Mohd Arbas        | - | 21E41A3314 |
| P.S. Jyothi       | - | 22E45A3301 |

Under the supervision of  
**Puneet Shetteppanavar , BE, M-Tech, PhD(Scholar)**

**Designation: Assistant Professor.**

**Department of Computer Science and Engineering**



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**  
**SREE DATTHA INSTITUTE OF ENGINEERING & SCIENCE**  
**AUTONOMOUS**

(Approved by AICTE New Delhi, Accredited by NAAC, Affiliate to JNTUH)

**SHERIGUDA (v), IBRAHIMPATNAM (M), RANGAREDDY -501510**

**2024-2025**

**SREE DATTHA INSTITUTE OF ENGINEERING AND SCIENCE**  
**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



**DECLARATION**

We are hereby declaring that the project report titled “**PREDICTING HOME PRICES WITH REGRESSION ANALYSIS USING THE BOSTON HOUSING DATASET**” under the guidance of **Puneet Shetteppanavar, Sree Dattha Institute of Engineering and Science**, Ibrahimpatnam is submitted in partial fulfillment of the requirement for the award of B. Tech. in Computer Science and Engineering is a record of bonafide work carried out by me and the results embodied in this project have not been reproduced or copied from any source.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

**Name of the Students**

|                   |            |
|-------------------|------------|
| Md. Ammaar Quadri | 21E41A3312 |
| M. Sahithi        | 21E41A3313 |
| Mohd Arbas        | 21E41A3314 |
| P.S. Jyothi       | 22E45A3301 |

**SREE DATTHA INSTITUTE OF ENGINEERING AND SCIENCE**  
**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



**CERTIFICATE**

This is to certify that the project entitled “**PREDICTING HOME PRICES WITH REGRESSION ANALYSIS USING THE BOSTON HOUSING DATASET**” is being submitted by **Md. Ammaar Quadri (21E41A3312), M. Sahithi (21E41A3313), Mohd Arbas (21E41A3314), P.S. Jyothi (22E45A3301)** in partial fulfillment of the requirements for the award of B. Tech IV year, I semester in Computer Science and Engineering to the Jawaharlal Nehru Technological University Hyderabad, is a record of Bonafide work carried out by them under our guidance and supervision during the academic year 2024-25.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Puneet Shetteppanavar

**Internal Guide**

Dr. SK. Mahaboob Basha

**HOD**

**External Examiner**

Submitted for viva Voice Examination held on \_\_\_\_\_

## ACKNOWLEDGEMENT

Apart from our efforts, the success of any project depends largely on the encouragement and guidelines of many others. We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project.

We would like to express our sincere gratitude to Chairman **Sri. G. Panduranga Reddy**, and Vice-Chairman **Dr. GNV Vibhav Reddy** for providing excellent infrastructure and a nice atmosphere throughout this project. We are obliged to **Dr. S. Venkata Achuta Rao**, Principal, Professor CSE & Dean Academics for providing encouragement and support for completing this project successfully.

We take this opportunity to express my profound gratitude and deep regard for my HOD **Dr. SK. Mahaboob Basha** & Internal guide **Dr. B Nageswara Rao**, Assistant Professor for his exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by him shall carry us a long way in the journey of life on which we are about to embark.

The guidance and support were received from all the members of Sree Dattha Institute of Engineering and Science who contributed to the completion of the project. We are grateful for their constant support and help.

Finally, we would like to take this opportunity to thank our family for their constant encouragement, without which this assignment would not be completed. We sincerely acknowledge and thank all those who gave support directly and indirectly in the completion of this project.

# **PREDICTING HOME PRICES: A BEGINNER'S JOURNEY WITH REGRESSION ANALYSIS USING THE BOSTON HOUSING DATASET**

## **ABSTRACT**

In the realm of real estate, accurately predicting home prices is a crucial task for buyers, sellers, and real estate professionals alike. It enables informed decision-making, aids in property valuation, and contributes to market analysis. Leveraging machine learning techniques for this purpose has become increasingly popular due to their ability to analyze complex data patterns and generate predictive models. In the traditional real estate market, pricing homes often relies on the expertise of real estate agents and appraisers, coupled with market trends and comparable sales data. While effective to some extent, this approach has limitations. It can be subjective, prone to human error, and may not fully capture all relevant factors affecting home prices. Moreover, it might not scale well for large datasets or provide insights into underlying data patterns. There is a growing need for more accurate, data-driven methods to predict home prices. With the advent of big data and machine learning, there is an opportunity to leverage advanced analytical techniques to enhance price prediction accuracy, improve decision-making, and provide valuable insights into the housing market. The challenge of predicting home prices accurately using regression analysis techniques. It seeks to build machine learning models that can effectively learn from historical housing data and make reliable predictions about the prices of new properties based on their features. The goal is to develop models that outperform traditional methods and provide actionable insights for stakeholders in the real estate industry. The proposed system involves leveraging the Boston Housing Dataset, a classic dataset widely used in machine learning research, to train and evaluate regression models. The project will employ various regression algorithms, such as Random Forest Regression and XGBoost Regression, to build predictive models. These models will be trained on features such as crime rate, zoning, proximity to employment centers, and others to predict home prices.

# INTRODUCTION

# CHAPTER 1

## INTRODUCTION

### 1.1 History

Accurately predicting the value of a plot or house is an important task for many house owners, house buyers, plot owners, plot buyers or stake holders. Real estate agencies and people buy and sell houses all the time, people buy houses to live in or as an investment whereas real estate agencies buy it to run a business. But the problem arises in evaluation of the cost of the property. Over-validation / Under-validation have always been the issues faced in house markets due to lack of proper detection measures. It is also very difficult task. We know that features like size, area, location etc affect the price of the property but there are many other features also which affect the property such as inflation rates in market, age of the property etc. In order to overcome these problems a throw analysis is done using Machine Learning (ML) which is a branch of Artificial Intelligence (AI). With the advancement of science and technology our daily life has become much easier. In today's world we use information and communication technology extensively. Every day a new technology emerges in our current digital age which improves the living standard of people. Sometimes these new technologies have negative effect but most of the times these technologies have positive effect [1]. AI or more widely known as AI is one of such technology which has improved the living standard of people [2] worldwide. AI is widely used now days in various fields like healthcare [3], real estate [4], stock market prediction [5], weather prediction [6], automobile [7] and also in many other fields [8][9]. AI has many subfields like Natural Language Processing, Machine Vision, Robotics, Expert System etc [10], but in this study ML is used. ML is a branch of AI which deals with certain tasks using past data or recorded data and various algorithms. These tasks of ML involve classification, association, clustering and regression. ML can be used to make predictive models to make predictions for future or can be used to make descriptive models to make acquire some kind of knowledge from the given data. The main difference between ML programming and conventional programming is that, in conventional programming, programs are created manually by providing input data and based on the programming logic computer generates the output. However, in ML, the inputs and the outputs are fed into the algorithm creating the program. ML approaches are mainly divided into three categories these are Reinforcement Learning, Unsupervised Learning and Supervised Learning. In supervised learning the computers are given inputs and

their desired outputs by a supervisor and the goal is to create a general rule using which a given input can be mapped into their desired output. In unsupervised learning there is no supervisor and the structures in the input are found on its own. The main goal of unsupervised learning is to find hidden structures or a mean towards an end. In reinforcement learning, in presence of a dynamic environment certain goals (such as driving a vehicle or playing against an opponent) are performed by the program. Using the feedback provided to the program as it navigates the problem space it is given reward and the goal is to maximize the rewards. Here machine is trained using various ML algorithms in Boston house dataset to create various models and using this trained machine models evaluation is done.

## **1.2 Research Motivation**

The motivation behind research in predicting home prices stems from the inherent importance of accurate price predictions in the real estate market. For buyers, sellers, and real estate professionals alike, the ability to predict home prices accurately is crucial for making informed decisions, assessing property values, and conducting market analysis. Traditional methods of home price prediction, relying heavily on subjective assessments and historical sales data, have limitations in terms of accuracy and scalability.

The advent of big data and machine learning technologies has presented an opportunity to overcome these limitations and develop more accurate, data-driven methods for predicting home prices. By leveraging advanced analytical techniques, researchers aim to enhance prediction accuracy, improve decision-making processes, and provide valuable insights into housing market dynamics. The growing availability of housing market data, coupled with advancements in computational power, has further fueled research efforts in this domain.

## **1.3 Problem Statement**

The challenge of predicting home prices accurately using regression analysis techniques lies in developing machine learning models that can effectively learn from historical housing data and make reliable predictions about the prices of new properties based on their features. Traditional methods of home price prediction, while effective to some extent, may not fully capture all relevant factors influencing home prices and are prone to subjective biases and human error.

The goal of research in this area is to address these challenges by building predictive models that outperform traditional methods and provide actionable insights for stakeholders in the



real estate industry. This involves leveraging large datasets, such as the Boston Housing Dataset, to train and evaluate regression models using various algorithms, such as Random Forest Regression and XGBoost Regression. By identifying and quantifying the relationships between housing market variables and property values, researchers aim to develop models capable of accurately forecasting home prices and uncovering valuable insights into market dynamics.

## **1.4 Applications**

The applications of predicting home prices accurately using regression analysis techniques presented in bullet points:

### **— Buyers:**

- Make informed decisions about purchasing properties by understanding their estimated values.
- Identify properties that offer the best value for money based on accurate price predictions.
- Plan their budget more effectively by knowing the expected price range of homes in desired locations.

### **— Sellers:**

- Set competitive listing prices for their properties to attract potential buyers.
- Maximize returns on their investments by pricing properties accurately.
- Negotiate effectively with buyers by having a clear understanding of the market value of their properties.

### **— Real Estate Professionals:**

- Assess property values accurately to provide reliable advice to clients.
- Develop effective pricing strategies for properties based on market trends and data-driven insights.
- Conduct market analysis to identify emerging trends and opportunities for investment.

### **— Financial Institutions:**

- Assess the risk associated with mortgage lending by accurately predicting property values.
- Make informed decisions about loan approvals and interest rates based on reliable price predictions.
- Manage mortgage portfolios more effectively by understanding the market dynamics of property values.

— **Insurance Companies:**

- Determine insurance premiums for properties based on accurate price predictions.
- Assess the risk of property damage and calculate insurance payouts accordingly.
- Develop pricing strategies for property insurance products to remain competitive in the market.

— **Government Agencies:**

- Inform housing policies and urban planning initiatives based on accurate price predictions.
- Identify areas of high demand for affordable housing and allocate resources accordingly.
- Support infrastructure development projects by understanding the impact of property values on local economies.

— **Investors:**

- Make informed decisions about real estate investments by predicting future property values.
- Identify investment opportunities in emerging markets or undervalued properties.
- Optimize investment portfolios by diversifying across properties with different expected returns.

## **CHAPTER 2**

### **LITERATURE SURVEY**

A lot of past works have been done for predicting house prices. Different levels of accuracies and results have been achieved using different methodologies, techniques and datasets. A study of independent real estate market forecasting on house price using data mining techniques was done by Bahia [11]. Here the main idea was to construct the neural network model using two types of neural network. The first one is Feed Forward Neural Network (FFNN) and the second one is Cascade Forward Neural Network 2022 2nd International Conference on Intelligent Technologies (CONIT) Karnataka, India. June 24-26, 2022 (CFNN). It was observed that CFNN gives a better result compared to FFNN using MSE performance metric. Mu et al. [12] did an analysis of dataset containing Boston suburb house values using several ML methods which are Support Vector Machine (SVM), Least Square Support Vector Machine (LSSVM) and Partial Least Square (PLS) methods. SVM and LSSVM gives superior performance compared to PLS. Beracha et al. [13] proved that high amenity areas experience greater price volatility by investigating the correlation between house prices volatility, returns and local amenities. Law [14] finds that there is a strong link between house price and street based local area compare to the house price and region based local area. Binbin et al. [15] to study London house price build a Geographically Weighted Regression (GWR) model considering Euclidean distance, travel time metrics and Road network distance. Marco et al. [16] to reduce the prediction errors, a mixed Geographically weighted regression(GWR) model is used that emphasize the importance and complex of the spatial Heterogeneity in Australia. Using State level data in USA, Sean et al. [17] have examined the correlation among common shocks, real per capita disposable income, house prices, net borrowing cost and macroeconomic, spatial factors and local disturbances and state level population growth. Joep et al. [18] using the administrative data from the Netherlands have found that wealthy buyers and high income leads to higher purchase price and wealthy sealer and higher income leads to lower selling price.

## CHAPTER 3

### EXISTING SYSTEM

#### 3.1 Random Forest Algorithm

Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique. It can be used for both Classification and Regression problems in ML. It is based on the concept of ensemble learning, which is a process of combining multiple classifiers to solve a complex problem and to improve the performance of the model. As the name suggests, "Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset." Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output. The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

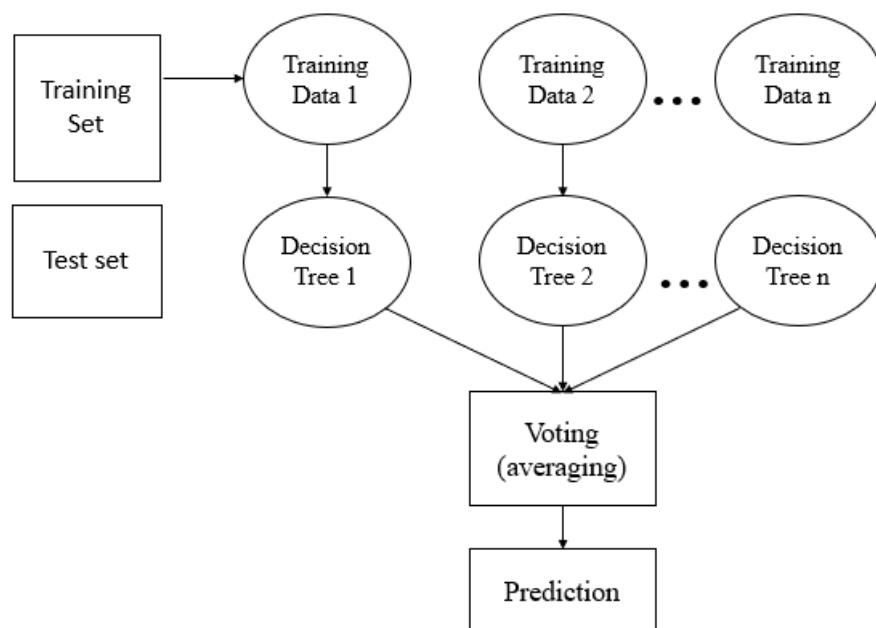


Fig. 1: Random Forest algorithm.

##### 3.1.1 Random Forest algorithm

Step 1: In Random Forest n number of random records are taken from the data set having k number of records.

Step 2: Individual decision trees are constructed for each sample.

Step 3: Each decision tree will generate an output.

Step 4: Final output is considered based on Majority Voting or Averaging for Classification and regression respectively.

### 3.1.2 Important Features of Random Forest

- **Diversity**- Not all attributes/variables/features are considered while making an individual tree, each tree is different.
- **Immune to the curse of dimensionality**- Since each tree does not consider all the features, the feature space is reduced.
- **Parallelization**-Each tree is created independently out of different data and attributes. This means that we can make full use of the CPU to build random forests.
- **Train-Test split**- In a random forest we don't have to segregate the data for train and test as there will always be 30% of the data which is not seen by the decision tree.
- **Stability**- Stability arises because the result is based on majority voting/ averaging.

### 3.1.3 Assumptions for Random Forest

Since the random forest combines multiple trees to predict the class of the dataset, it is possible that some decision trees may predict the correct output, while others may not. But together, all the trees predict the correct output. Therefore, below are two assumptions for a better Random Forest classifier:

- There should be some actual values in the feature variable of the dataset so that the classifier can predict accurate results rather than a guessed result.
- The predictions from each tree must have very low correlations.

Below are some points that explain why we should use the Random Forest algorithm

- It takes less training time as compared to other algorithms.
- It predicts output with high accuracy, even for the large dataset it runs efficiently.
- It can also maintain accuracy when a large proportion of data is missing.

### 3.1.4 Types of Ensembles

Before understanding the working of the random forest, we must look into the ensemble technique. Ensemble simply means combining multiple models. Thus, a collection of models is used to make predictions rather than an individual model. Ensemble uses two types of methods:

**Bagging**– It creates a different training subset from sample training data with replacement & the final output is based on majority voting. For example, Random Forest. Bagging, also known as Bootstrap Aggregation is the ensemble technique used by random forest. Bagging chooses a random sample from the data set. Hence each model is generated from the samples (Bootstrap Samples) provided by the Original Data with replacement known as row sampling. This step of row sampling with replacement is called bootstrap. Now each model is trained independently which generates results. The final output is based on majority voting after combining the results of all models. This step which involves combining all the results and generating output based on majority voting is known as aggregation.

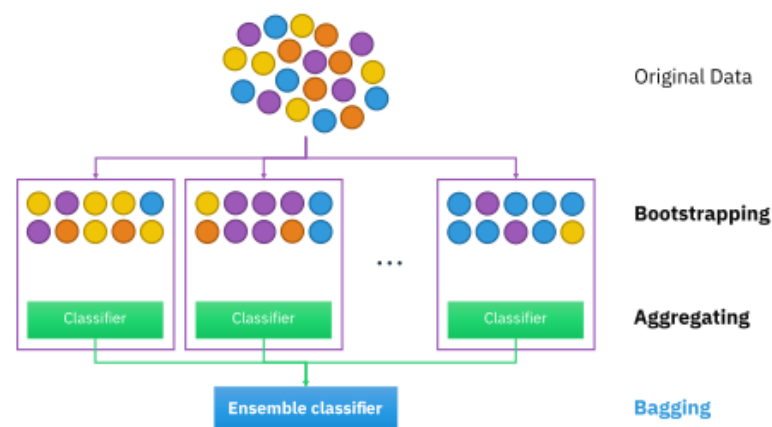


Fig. 2: RF Classifier analysis.

**Boosting**– It combines weak learners into strong learners by creating sequential models such that the final model has the highest accuracy. For example, ADA BOOST, XG BOOST.

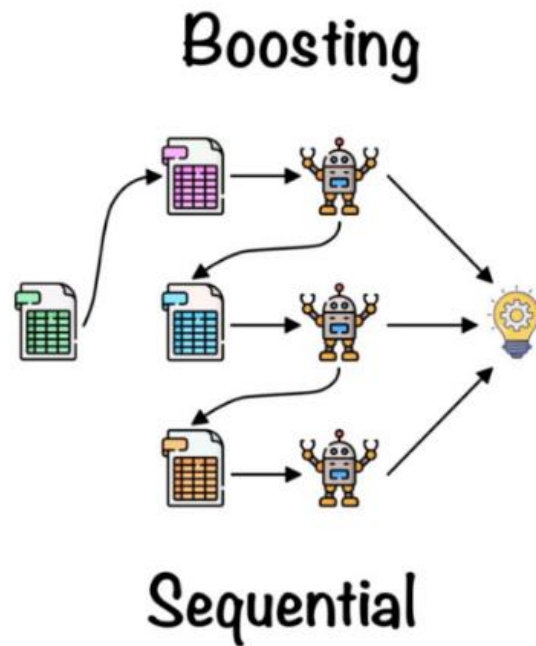


Fig. 3: Boosting RF Classifier.

### 3.2 Disadvantages

While Random Forest is a powerful and versatile machine learning algorithm, it does come with some disadvantages:

- **Lack of Interpretability:** Random Forest models are often considered as "black boxes" because it's challenging to interpret the individual trees in the forest, especially when the forest becomes large and complex.
- **Memory and Computation:** Random Forest models can become memory-intensive and computationally expensive, especially when dealing with a large number of trees or features. This can be a limitation for real-time prediction tasks or when working with large datasets.
- **Overfitting:** Although Random Forests are less prone to overfitting compared to individual decision trees, they can still overfit noisy data if not properly tuned. Tuning parameters such as the number of trees and the maximum depth of trees is essential to mitigate this risk.
- **Not Suitable for Imbalanced Data:** Random Forest may not perform well on highly imbalanced datasets where one class is significantly more frequent than the others. It tends to favor the majority class, leading to biased predictions.

- **Training Time:** Training a Random Forest model can take longer compared to simpler algorithms like linear regression or decision trees, especially when dealing with large datasets or a high number of trees.
- **Limited Extrapolation Ability:** Random Forest models may struggle with making predictions outside the range of the training data. They may not generalize well to unseen data points that are significantly different from those in the training set.



# CHAPTER 4

## PROPOSED SYSTEM

### 4.1 Overview

The project aims to predict home prices using machine learning techniques applied to the Boston Housing Dataset. Here's an overview of the project code explanation:

#### — Data Loading and Preprocessing:

- The code begins by importing necessary libraries such as Pandas, NumPy, Matplotlib, Seaborn, scikit-learn, XGBoost. It loads the dataset from a CSV file using Pandas' `read_csv()` function. Exploratory Data Analysis (EDA) techniques are applied to understand the dataset, including checking data types, summary statistics, and identifying missing values. Visualization techniques like heatmaps, pairplots, and histograms are used to gain insights into data distributions and relationships between variables.

#### — Data Splitting:

- The dataset is split into features (independent variables) and the target variable (home prices). The train-test split is performed using scikit-learn's `train_test_split()` function, allocating a certain percentage (e.g., 80%) of the data for training and the remaining for testing.

#### — Model Training:

- Two regression models, `RandomForestRegressor` and `XGBoost`, are trained using the training data. For `RandomForestRegressor`, the code checks if a pre-trained model exists; if not, it initializes and fits a new model to the training data. Similarly, for `XGBoost`, the code either loads a pre-trained model or trains a new one. The models are trained to learn the relationships between the input features and home prices in the training data.

#### — Model Evaluation:

- A function called `performance_metrics()` is defined to evaluate the performance of each regression model. This function computes metrics such

as Mean Squared Error (MSE), Mean Absolute Error (MAE), and R-squared ( $R^2$ ) score to assess the model's accuracy. Additionally, it visualizes the predicted vs. true values using a scatter plot to visually inspect the model's performance.

— **Model Prediction:**

- Once trained, the models are used to make predictions on the test data.
- The predictions are compared against the actual home prices in the test data to assess the models' generalization performance.

— **Prediction on New Data:**

- Lastly, the code reads a separate testing dataset (new data) from a CSV file. The trained XGBoost model is used to predict home prices on this new dataset. The predictions are appended to the testing dataset for further analysis or deployment.

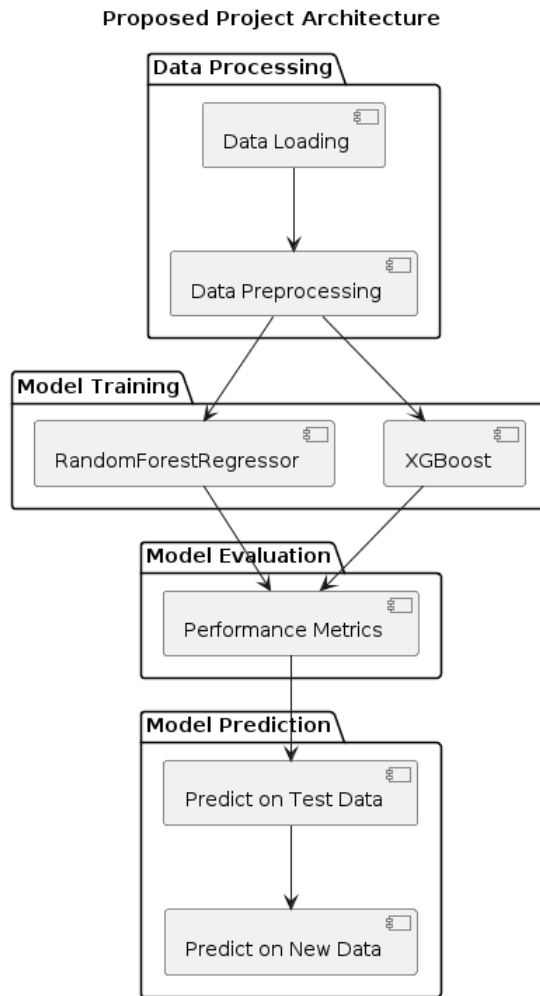


Figure 4.1: Block Diagram of Proposed System.

## 4.2 Data Preprocessing

Data pre-processing is a process of preparing the raw data and making it suitable for a machine learning model. It is the first and crucial step while creating a machine learning model. When creating a machine learning project, it is not always a case that we come across the clean and formatted data. And while doing any operation with data, it is mandatory to clean it and put in a formatted way. So, for this, we use data pre-processing task. A real-world data generally contains noises, missing values, and maybe in an unusable format which cannot be directly used for machine learning models. Data pre-processing is required tasks for cleaning the data and making it suitable for a machine learning model which also increases the accuracy and efficiency of a machine learning model.

- Getting the dataset
- Importing libraries

- Importing datasets
- Finding Missing Data
- Encoding Categorical Data
- Splitting dataset into training and test set

**Importing Libraries:** To perform data preprocessing using Python, we need to import some predefined Python libraries. These libraries are used to perform some specific jobs. There are three specific libraries that we will use for data preprocessing, which are:

**Numpy:** Numpy Python library is used for including any type of mathematical operation in the code. It is the fundamental package for scientific calculation in Python. It also supports to add large, multidimensional arrays and matrices. So, in Python, we can import it as:

```
import numpy as nm
```

Here we have used nm, which is a short name for Numpy, and it will be used in the whole program.

**Matplotlib:** The second library is matplotlib, which is a Python 2D plotting library, and with this library, we need to import a sub-library pyplot. This library is used to plot any type of charts in Python for the code. It will be imported as below:

```
import matplotlib.pyplot as mtp
```

Here we have used mtp as a short name for this library.

**Pandas:** The last library is the Pandas library, which is one of the most famous Python libraries and used for importing and managing the datasets. It is an open-source data manipulation and analysis library. Here, we have used pd as a short name for this library. Consider the below image:

```
1 # importing libraries
2 import numpy as nm
3 import matplotlib.pyplot as mtp
4 import pandas as pd
5
```

**Handling Missing data:** The next step of data preprocessing is to handle missing data in the datasets. If our dataset contains some missing data, then it may create a huge problem for our

machine learning model. Hence it is necessary to handle missing values present in the dataset. There are mainly two ways to handle missing data, which are:

- By deleting the particular row: The first way is used to commonly deal with null values. In this way, we just delete the specific row or column which consists of null values. But this way is not so efficient and removing data may lead to loss of information which will not give the accurate output.
- By calculating the mean: In this way, we will calculate the mean of that column or row which contains any missing value and will put it on the place of missing value. This strategy is useful for the features which have numeric data such as age, salary, year, etc.

**Encoding Categorical data:** Categorical data is data which has some categories such as, in our dataset; there are two categorical variables, Country, and Purchased. Since machine learning model completely works on mathematics and numbers, but if our dataset would have a categorical variable, then it may create trouble while building the model. So, it is necessary to encode these categorical variables into numbers.

### 4.3 Splitting the Dataset

In machine learning data preprocessing, we divide our dataset into a training set and test set. This is one of the crucial steps of data preprocessing as by doing this, we can enhance the performance of our machine learning model. Suppose if we have given training to our machine learning model by a dataset and we test it by a completely different dataset. Then, it will create difficulties for our model to understand the correlations between the models. If we train our model very well and its training accuracy is also very high, but we provide a new dataset to it, then it will decrease the performance. So we always try to make a machine learning model which performs well with the training set and also with the test dataset. Here, we can define these datasets as:

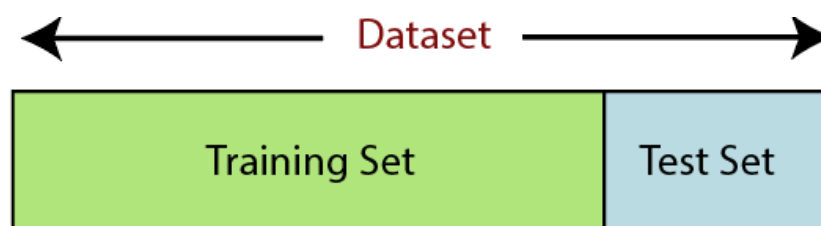


Figure 4.2: Splitting the dataset.

**Training Set:** A subset of dataset to train the machine learning model, and we already know the output.

**Test set:** A subset of dataset to test the machine learning model, and by using the test set, model predicts the output.

For splitting the dataset, we will use the below lines of code:

```
from sklearn.model_selection import train_test_split  
  
x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.2, random_state=0)
```

### Explanation

- In the above code, the first line is used for splitting arrays of the dataset into random train and test subsets.
- In the second line, we have used four variables for our output that are
  - x\_train: features for the training data
  - x\_test: features for testing data
  - y\_train: Dependent variables for training data
  - y\_test: Independent variable for testing data
- In train\_test\_split() function, we have passed four parameters in which first two are for arrays of data, and test\_size is for specifying the size of the test set. The test\_size maybe .5, .3, or .2, which tells the dividing ratio of training and testing sets.
- The last parameter random\_state is used to set a seed for a random generator so that you always get the same result, and the most used value for this is 42.

## 4.4 XGBoost Model

XGBoost is a popular machine learning algorithm that belongs to the supervised learning technique. It can be used for both Classification and Regression problems in ML. It is based on the concept of ensemble learning, which is a process of combining multiple classifiers to solve a complex problem and to improve the performance of the model. As the name suggests, "XGBoost is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset."

Instead of relying on one decision tree, the XGBoost takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output. The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

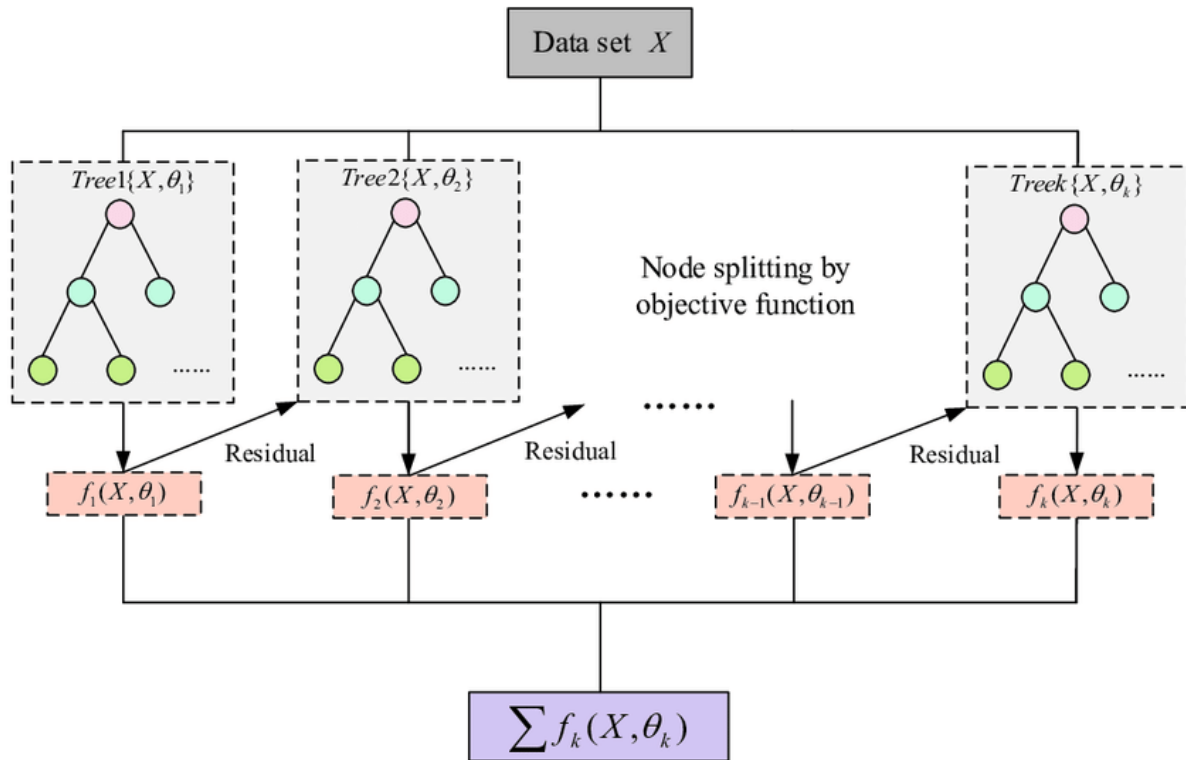


Fig. 4.2: XGBoost algorithm.

XGBoost, which stands for "Extreme Gradient Boosting," is a popular and powerful machine learning algorithm used for both classification and regression tasks. It is known for its high predictive accuracy and efficiency, and it has won numerous data science competitions and is widely used in industry and academia. Here are some key characteristics and concepts related to the XGBoost algorithm:

- **Gradient Boosting:** XGBoost is an ensemble learning method based on the gradient boosting framework. It builds a predictive model by combining the predictions of multiple weak learners (typically decision trees) into a single, stronger model.
- **Tree-based Models:** Decision trees are the weak learners used in XGBoost. These are shallow trees, often referred to as "stumps" or "shallow trees," which helps prevent overfitting.
- **Objective Function:** XGBoost uses a specific objective function that needs to be optimized during training. The objective function consists of two parts: a loss function

that quantifies the error between predicted and actual values and a regularization term to control model complexity and prevent overfitting. The most common loss functions are for regression (e.g., Mean Squared Error) and classification (e.g., Log Loss).

- **Gradient Descent Optimization:** XGBoost optimizes the objective function using gradient descent. It calculates the gradients of the objective function with respect to the model's predictions and updates the model iteratively to minimize the loss.
- **Regularization:** XGBoost provides several regularization techniques, such as L1 (Lasso) and L2 (Ridge) regularization, to control overfitting. These regularization terms are added to the objective function.
- **Parallel and Distributed Computing:** XGBoost is designed to be highly efficient. It can take advantage of parallel processing and distributed computing to train models quickly, making it suitable for large datasets.
- **Handling Missing Data:** XGBoost has built-in capabilities to handle missing data without requiring imputation. It does this by finding the optimal split for missing values during tree construction.
- **Feature Importance:** XGBoost provides a way to measure the importance of each feature in the model. This can help in feature selection and understanding which features contribute the most to the predictions.
- **Early Stopping:** To prevent overfitting, XGBoost supports early stopping, which allows training to stop when the model's performance on a validation dataset starts to degrade.
- **Scalability:** XGBoost is versatile and can be applied to a wide range of machine learning tasks, including classification, regression, ranking, and more.
- **Python and R Libraries:** XGBoost is available through libraries in Python (e.g., **xgboost**) and R (e.g., **xgboost**), making it accessible and easy to use for data scientists and machine learning practitioners.

XGBoost, which stands for eXtreme Gradient Boosting, is a popular machine learning algorithm that is particularly effective for structured/tabular data and is often used for tasks



like classification, regression, and ranking. It is an ensemble learning technique based on decision trees. Here's how XGBoost operates:

- **Ensemble Learning:** XGBoost is an ensemble learning method, which means it combines the predictions from multiple machine learning models to make more accurate predictions than any single model. It uses an ensemble of decision trees, known as "boosted trees."
- **Boosting:** Boosting is a sequential technique in which multiple weak learners (usually decision trees with limited depth) are trained one after the other. Each new tree tries to correct the errors made by the previous ones.
- **Gradient Boosting:** XGBoost is a gradient boosting algorithm. It minimizes a loss function by adding weak models (trees) that minimize the gradient of the loss function at each stage. This is done by fitting a tree to the residuals (the differences between the predicted and actual values) of the previous model.
- **Regularization:** XGBoost includes L1 (Lasso regression) and L2 (Ridge regression) regularization terms in the objective function to prevent overfitting. These regularization terms help control the complexity of individual trees and reduce the risk of overfitting the training data.
- **Tree Pruning:** XGBoost uses a technique called "pruning" to remove branches of the trees that do not contribute significantly to the model's predictive power. This reduces the complexity of the trees and helps prevent overfitting.
- **Feature Importance:** XGBoost provides a feature importance score, which helps you understand the contribution of each feature (input variable) in making predictions. You can use this information for feature selection and interpretation.
- **Parallel and Distributed Computing:** XGBoost is designed for efficiency and can take advantage of parallel and distributed computing to train on large datasets faster.
- **Handling Missing Data:** XGBoost can handle missing data by finding an optimal direction for missing values during tree construction.
- **Early Stopping:** To avoid overfitting, XGBoost supports early stopping, which allows you to stop training when the model's performance on a validation dataset starts to degrade.

- **Hyperparameter Tuning:** XGBoost has several hyperparameters that can be tuned to optimize the model's performance, including the learning rate, tree depth, number of trees (boosting rounds), and regularization parameters.

#### 4.5 Advantages

The proposed research work, which combines Edge Computing, Light Weight Homomorphic Encryption, and the XGBOOST classifier in a privacy-preserving healthcare application, offers several distinct advantages:

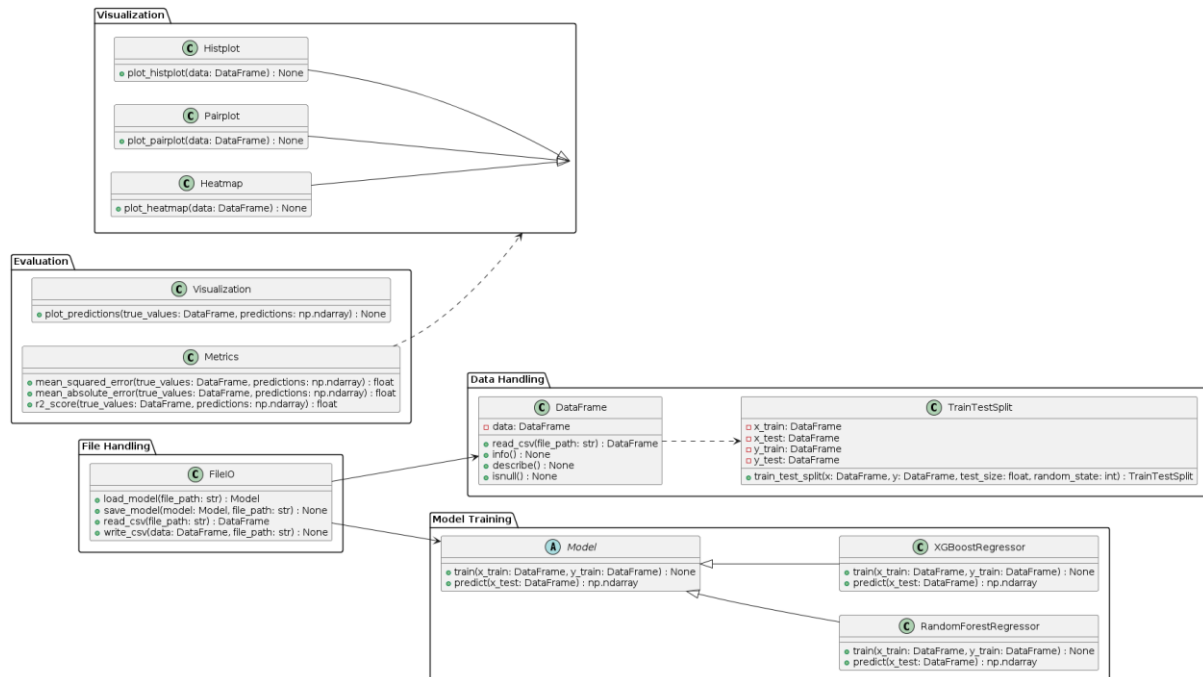
- **Enhanced Data Privacy:** One of the foremost advantages is the robust protection of patient data privacy. The use of Light Weight Homomorphic Encryption ensures that sensitive medical information remains confidential throughout the entire process, from data collection to disease prediction. This not only complies with stringent privacy regulations but also builds trust among patients, encouraging them to engage with healthcare applications more freely.
- **Reduced Response Times:** The introduction of Edge Nodes significantly reduces response times for disease prediction and diagnosis. By selecting the nearest and available Edge Node, the research minimizes latency, especially crucial in critical medical situations such as heart disease diagnosis. This enhancement in speed can lead to more timely interventions and improved patient outcomes.
- **Scalability and Efficiency:** The architecture's scalability is another advantage. As the system grows and accommodates more users and healthcare data, it can efficiently distribute tasks among multiple Edge Nodes. This scalability ensures that response times remain consistently low, even as the user base expands, making it a practical solution for large-scale healthcare applications.
- **Accurate Disease Prediction:** The use of the XGBOOST classifier, trained on encrypted data, ensures high accuracy in disease prediction. This accuracy is essential for healthcare professionals, as it aids in early detection and precise diagnosis, allowing for better treatment planning and patient care.
- **Real-World Applicability:** The research offers a practical solution that can be implemented in real-world healthcare settings. By addressing the critical issues of data privacy and response times, it provides healthcare providers and patients with a tool that enhances the quality and accessibility of healthcare services.

- **Interoperability:** The proposed framework can be integrated into existing healthcare systems and applications. It offers the flexibility to collaborate with various healthcare providers, ensuring that the benefits of privacy-preserving disease prediction can be leveraged across the healthcare industry.
- **Cross-Domain Potential:** Beyond healthcare, the principles of privacy-preserving data processing and edge computing have broader applicability. The research's architecture can serve as a model for other domains where data privacy and real-time processing are paramount, such as finance, telecommunications, and Internet of Things (IoT) applications.

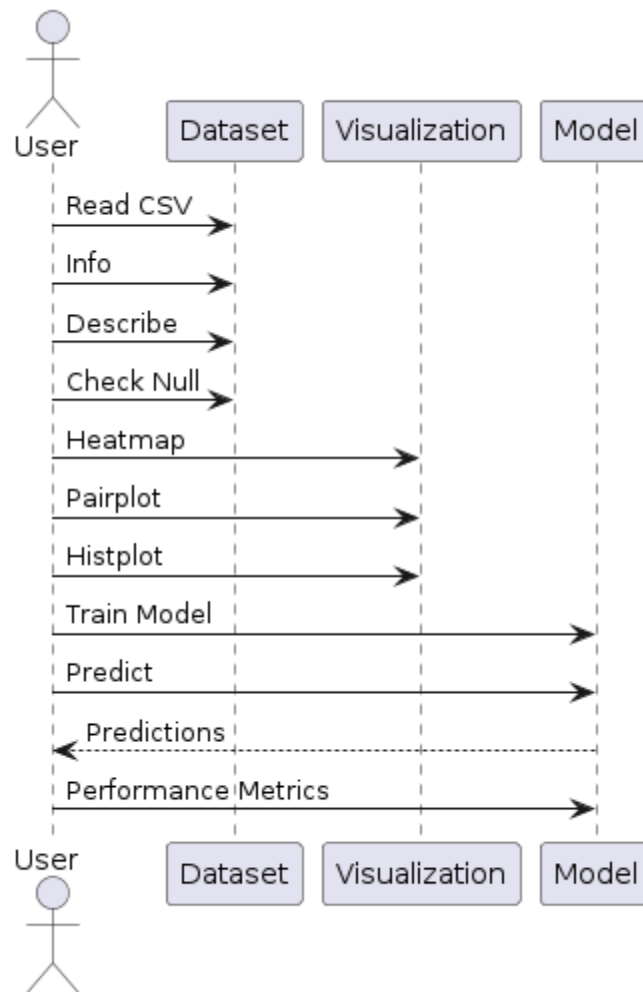
# CHAPTER 5

## UML DIAGRAMS

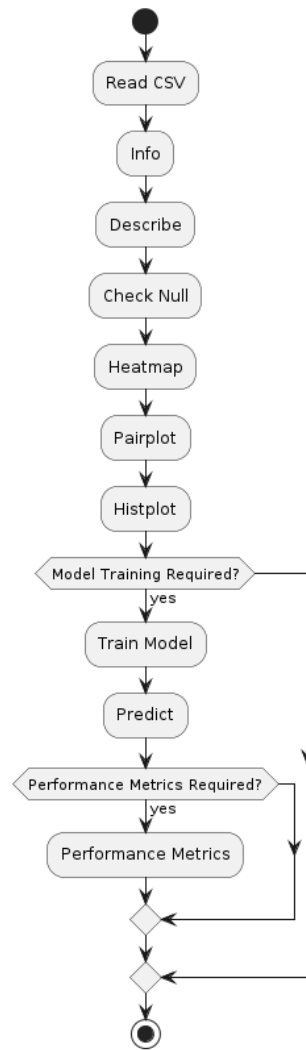
**Class Diagram:** Class diagram is a static diagram. It represents the static view of an application.



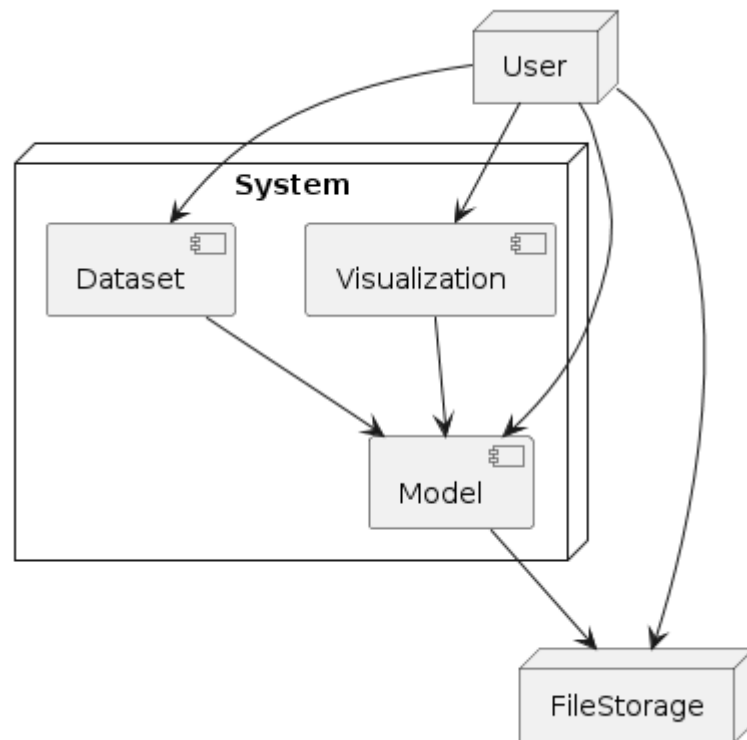
**Sequence Diagram:** Sequence diagram is an interaction diagram that details how operations are carried out.



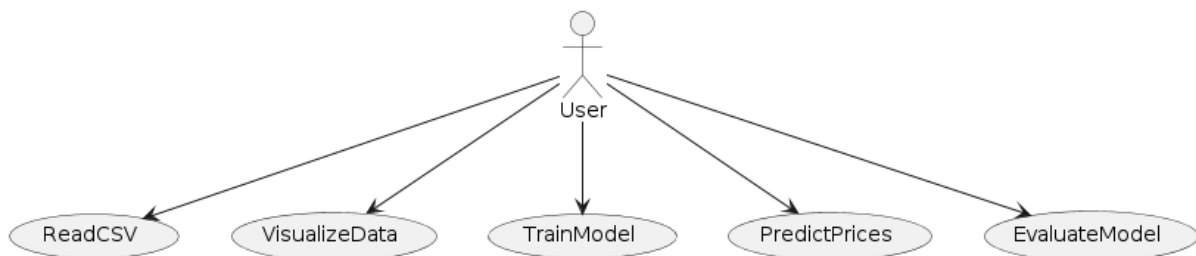
**Activity diagram:** Activity diagram is another important diagram in UML to *describe the dynamic aspects of the system*.



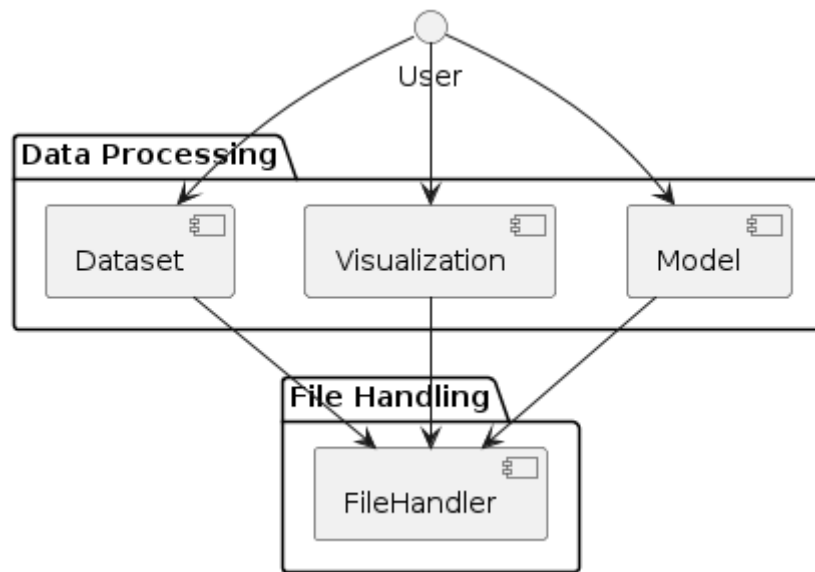
**Deployment diagram:** The deployment diagram *visualizes the physical hardware on which the software will be deployed.*



**Usecase diagram:** The purpose of use case diagram is to *capture the dynamic aspect of a system*.



**Component diagram:** Component diagram describes the organization and wiring of the physical components in a system.





# CHAPTER 6

## SOFTWARE ENVIRONMENT

### What is Python?

Below are some facts about Python.

- Python is currently the most widely used multi-purpose, high-level programming language.
- Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.
- Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.
- Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard library which can be used for the following –

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc. )
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

### Advantages of Python

Let's see how Python dominates over other languages.

#### 1. Extensive Libraries

Python downloads with an extensive library and it contains code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

## **2. Extensible**

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

## **3. Embeddable**

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

## **4. Improved Productivity**

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

## **5. IOT Opportunities**

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

## **6. Simple and Easy**

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

## **7. Readable**

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. This further aids the readability of the code.

## **8. Object-Oriented**

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

## **9. Free and Open-Source**

Like we said earlier, Python is freely available. But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

## **10. Portable**

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any system-dependent features.

## **11. Interpreted**

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.

Any doubts till now in the advantages of Python? Mention in the comment section.

## **Advantages of Python Over Other Languages**

### **1. Less Coding**

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

### **2. Affordable**

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.

### **3. Python is for Everyone**

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and machine learning, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

### **Disadvantages of Python**

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

#### **1. Speed Limitations**

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

#### **2. Weak in Mobile Computing and Browsers**

While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle.

The reason it is not so famous despite the existence of Brython is that it isn't that secure.

#### **3. Design Restrictions**

As you know, Python is dynamically typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.

#### **4. Underdeveloped Database Access Layers**

Compared to more widely used technologies like JDBC (Java DataBase Connectivity) and ODBC (Open DataBase Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

#### **5. Simple**

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

#### **Modules Used in Project**

##### **NumPy**

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary datatypes can be defined using NumPy which allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

##### **Pandas**

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python

with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

## **Matplotlib**

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and Ipython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with Ipython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

## **Scikit – learn**

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Python

## **Install Python Step-by-Step in Windows and Mac**

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

## **How to Install Python on Windows and Mac**

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python

but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your System Requirements. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a Windows 64-bit operating system. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheatsheet here. The steps on how to install Python on Windows 10, 8 and 7 are divided into 4 parts to help understand better.

### **Download the Correct version into the system**

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: <https://www.python.org>



Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.



Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

Looking for a specific release?

Python releases by version number:

| Release version               | Release date   |                          | Click for more                |
|-------------------------------|----------------|--------------------------|-------------------------------|
| <a href="#">Python 3.7.4</a>  | July 8, 2019   | <a href="#">Download</a> | <a href="#">Release Notes</a> |
| <a href="#">Python 3.6.9</a>  | July 2, 2019   | <a href="#">Download</a> | <a href="#">Release Notes</a> |
| <a href="#">Python 3.7.3</a>  | March 25, 2019 | <a href="#">Download</a> | <a href="#">Release Notes</a> |
| <a href="#">Python 3.4.10</a> | March 18, 2019 | <a href="#">Download</a> | <a href="#">Release Notes</a> |
| <a href="#">Python 3.5.7</a>  | March 18, 2019 | <a href="#">Download</a> | <a href="#">Release Notes</a> |
| <a href="#">Python 2.7.16</a> | March 4, 2019  | <a href="#">Download</a> | <a href="#">Release Notes</a> |
| <a href="#">Python 3.7.2</a>  | Dec. 24, 2018  | <a href="#">Download</a> | <a href="#">Release Notes</a> |

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.



| Files                                               |                  |                             |                                  |           |                     |
|-----------------------------------------------------|------------------|-----------------------------|----------------------------------|-----------|---------------------|
| Version                                             | Operating System | Description                 | MD5 Sum                          | File Size | GPG                 |
| <a href="#">Gzipped source tarball</a>              | Source release   |                             | 68111671e5b2db4ae7b9ab01b0f09be  | 23017663  | <a href="#">SIG</a> |
| <a href="#">XZ compressed source tarball</a>        | Source release   |                             | d33e4aae6097051c2eca45ee3604803  | 17131432  | <a href="#">SIG</a> |
| <a href="#">macOS 64-bit/32-bit installer</a>       | Mac OS X         | for Mac OS X 10.6 and later | 6428b4fa7583da91a440c8a1cee08e6  | 34898416  | <a href="#">SIG</a> |
| <a href="#">macOS 64-bit installer</a>              | Mac OS X         | for OS X 10.9 and later     | 5dd805c38217a457738f5e4a936b243f | 20082845  | <a href="#">SIG</a> |
| <a href="#">Windows .help file</a>                  | Windows          |                             | d63999573a2c96b2ac56cade6b477cd2 | 8131761   | <a href="#">SIG</a> |
| <a href="#">Windows x86-64 embeddable zip file</a>  | Windows          | for AMD64/EM64T/x64         | 98093cfd9ec0b0a6e3184a4072ba2    | 7504391   | <a href="#">SIG</a> |
| <a href="#">Windows x86-64 executable installer</a> | Windows          | for AMD64/EM64T/x64         | a702b4b0ad76dbdb3543a383e543400  | 26880368  | <a href="#">SIG</a> |
| <a href="#">Windows x86-64 web-based installer</a>  | Windows          | for AMD64/EM64T/x64         | 28cb1c608b6d73a8e53a3bd353b4bd2  | 1382904   | <a href="#">SIG</a> |
| <a href="#">Windows x86 embeddable zip file</a>     | Windows          |                             | 9fab3b818841879fda94113574139d8  | 6741626   | <a href="#">SIG</a> |
| <a href="#">Windows x86 executable installer</a>    | Windows          |                             | 31cc802942a5446a3d0451476394789  | 25663848  | <a href="#">SIG</a> |
| <a href="#">Windows x86 web-based installer</a>     | Windows          |                             | 1b670cfa5d317df82c30983ea371d87c | 1324608   | <a href="#">SIG</a> |

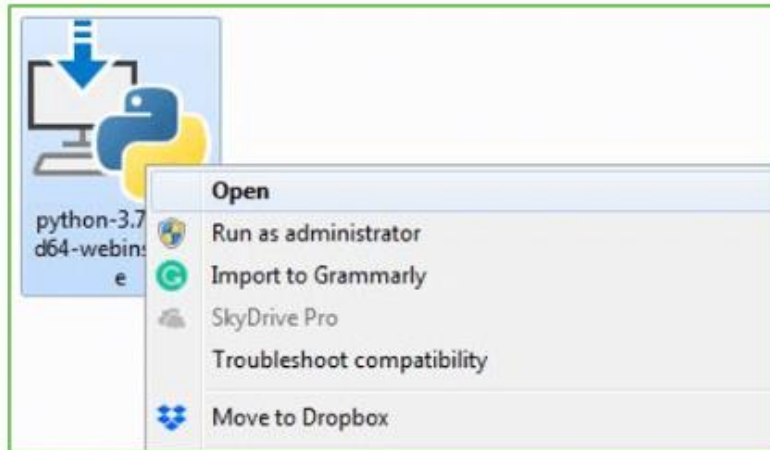
- To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.
- To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

## Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH.



Step 3: Click on Install NOW After the installation is successful. Click on Close.



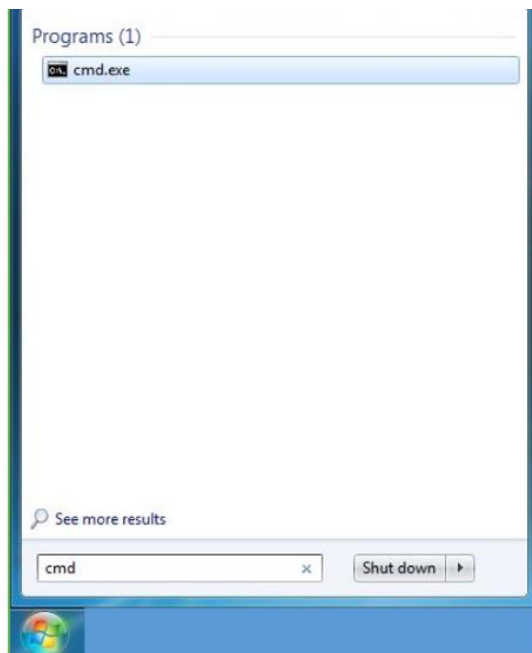
With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

### Verify the Python Installation

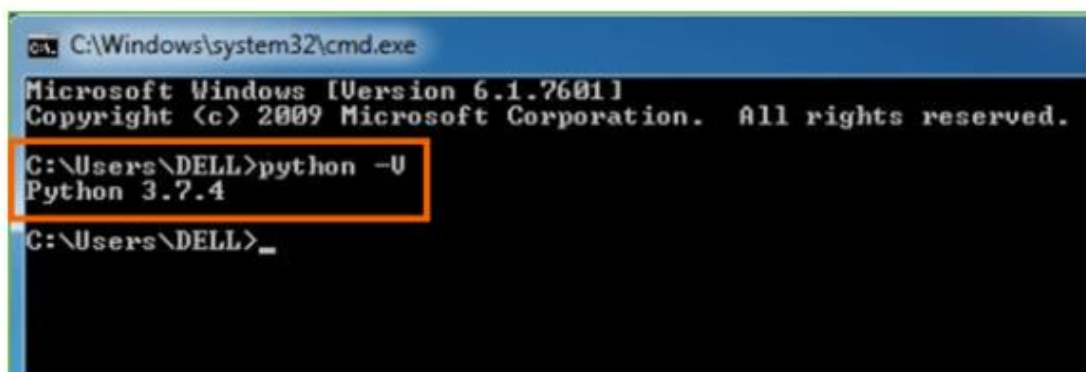
Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type python -V and press Enter.



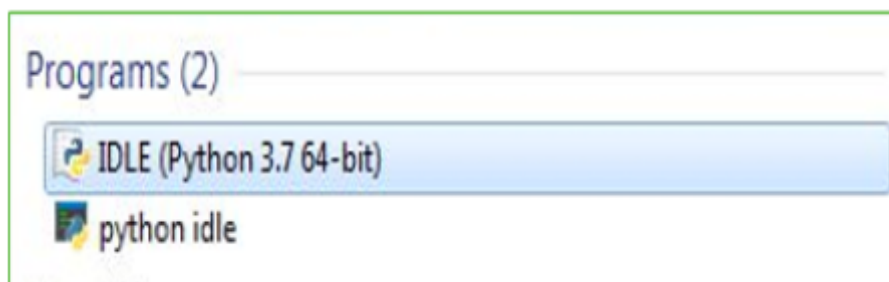
Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

### Check how the Python IDLE works

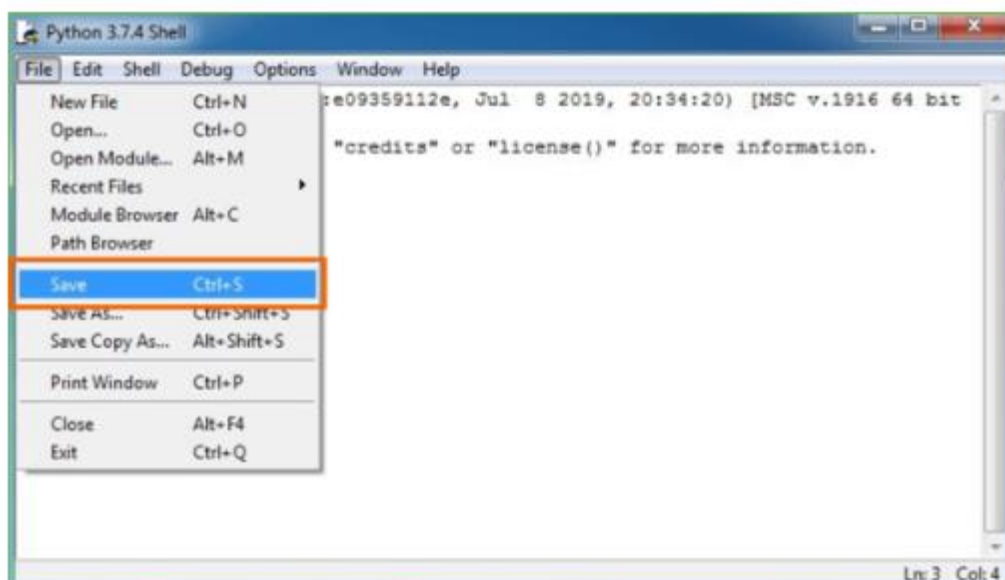
Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.



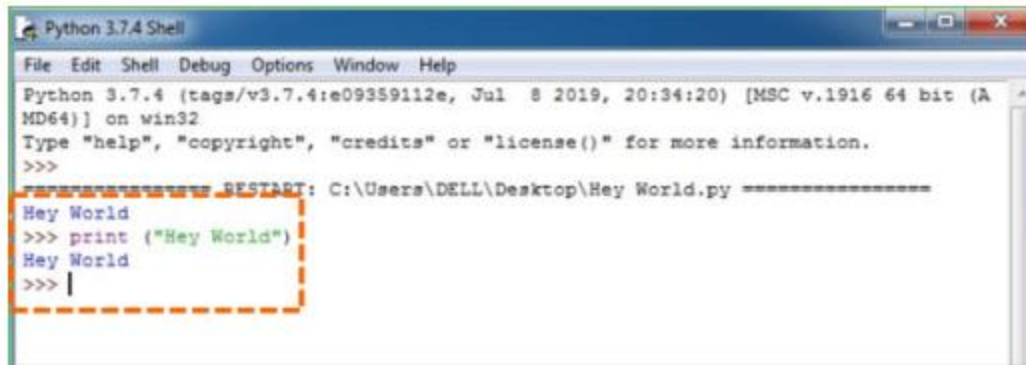
Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. Click on File > Click on Save



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. enter print (“Hey World”) and Press Enter.

A screenshot of a Windows command prompt window titled "Python 3.7.4 Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The text inside the window shows the Python version and build information: "Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (AMD64)] on win32". It then displays the help message: "Type 'help', 'copyright', 'credits' or 'license()' for more information." followed by ">>>". Below this, a line of text indicates a file restart: "\*\*\*\*\* RESTART: C:\Users\DELL\Desktop\Hey World.py \*\*\*\*\*". The output of the script is shown: "Hey World". Then, the user enters the command ">>> print ('Hey World')", and the output "Hey World" is displayed. The prompt ">>> |" is shown at the end of the line. A dashed orange box highlights the output "Hey World" and the command ">>> print ('Hey World')".

```
Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (A
MD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
***** RESTART: C:\Users\DELL\Desktop\Hey World.py *****
Hey World
>>> print ('Hey World')
Hey World
>>> |
```

You will see that the command given is launched. With this, we end our tutorial on how to install Python. You have learned how to download python for windows into your respective operating system.

Note: Unlike Java, Python does not need semicolons at the end of the statements otherwise it won't work.

# CHAPTER 7

## SYSTEM REQUIREMENTS

### Software Requirements

The functional requirements or the overall description documents include the product perspective and features, operating system and operating environment, graphics requirements, design constraints and user documentation.

The appropriation of requirements and implementation constraints gives the general overview of the project in regard to what the areas of strength and deficit are and how to tackle them.

- Python IDLE 3.7 version (or)
- Anaconda 3.7 (or)
- Jupiter (or)
- Google colab

### Hardware Requirements

Minimum hardware requirements are very dependent on the particular software being developed by a given Enthought Python / Canopy / VS Code user. Applications that need to store large arrays/objects in memory will require more RAM, whereas applications that need to perform numerous calculations or tasks more quickly will require a faster processor.

- Operating system : Windows, Linux
- Processor : minimum intel i3
- Ram : minimum 4 GB
- Hard disk : minimum 250GB

# **CHAPTER 8**

## **FUNCTIONAL REQUIREMENTS**

### **Output Design**

Outputs from computer systems are required primarily to communicate the results of processing to users. They are also used to provide a permanent copy of the results for later consultation. The various types of outputs in general are:

- External Outputs, whose destination is outside the organization
- Internal Outputs whose destination is within organization and they are the
- User's main interface with the computer.
- Operational outputs whose use is purely within the computer department.
- Interface outputs, which involve the user in communicating directly.

### **Output Definition**

The outputs should be defined in terms of the following points:

- Type of the output
- Content of the output
- Format of the output
- Location of the output
- Frequency of the output
- Volume of the output
- Sequence of the output

It is not always desirable to print or display data as it is held on a computer. It should be decided as which form of the output is the most suitable.

### **Input Design**

Input design is a part of overall system design. The main objective during the input design is as given below:

- To produce a cost-effective method of input.
- To achieve the highest possible level of accuracy.
- To ensure that the input is acceptable and understood by the user.

### **Input Stages**

The main input stages can be listed as below:

- Data recording
- Data transcription
- Data conversion
- Data verification
- Data control
- Data transmission
- Data validation
- Data correction

### **Input Types**

It is necessary to determine the various types of inputs. Inputs can be categorized as follows:

- External inputs, which are prime inputs for the system.
- Internal inputs, which are user communications with the system.
- Operational, which are computer department's communications to the system?
- Interactive, which are inputs entered during a dialogue.

### **Input Media**

At this stage choice has to be made about the input media. To conclude about the input media consideration has to be given to;



- Type of input
- Flexibility of format
- Speed
- Accuracy
- Verification methods
- Rejection rates
- Ease of correction
- Storage and handling requirements
- Security
- Easy to use
- Portability

Keeping in view the above description of the input types and input media, it can be said that most of the inputs are of the form of internal and interactive. As

Input data is to be the directly keyed in by the user, the keyboard can be considered to be the most suitable input device.

### **Error Avoidance**

At this stage care is to be taken to ensure that input data remains accurate from the stage at which it is recorded up to the stage in which the data is accepted by the system. This can be achieved only by means of careful control each time the data is handled.

### **Error Detection**

Even though every effort is made to avoid the occurrence of errors, still a small proportion of errors is always likely to occur, these types of errors can be discovered by using validations to check the input data.

### **Data Validation**

Procedures are designed to detect errors in data at a lower level of detail. Data validations have been included in the system in almost every area where there is a possibility for the user

to commit errors. The system will not accept invalid data. Whenever an invalid data is keyed in, the system immediately prompts the user and the user has to again key in the data and the system will accept the data only if the data is correct. Validations have been included where necessary.

The system is designed to be a user friendly one. In other words the system has been designed to communicate effectively with the user. The system has been designed with popup menus.

### **User Interface Design**

It is essential to consult the system users and discuss their needs while designing the user interface:

#### **User Interface Systems Can Be Broadly Clasified As:**

- User initiated interface the user is in charge, controlling the progress of the user/computer dialogue. In the computer-initiated interface, the computer selects the next stage in the interaction.
- Computer initiated interfaces

In the computer-initiated interfaces the computer guides the progress of the user/computer dialogue. Information is displayed and the user response of the computer takes action or displays further information.

#### **User Initiated Interfaces**

User initiated interfaces fall into two approximate classes:

- Command driven interfaces: In this type of interface the user inputs commands or queries which are interpreted by the computer.
- Forms oriented interface: The user calls up an image of the form to his/her screen and fills in the form. The forms-oriented interface is chosen because it is the best choice.

#### **Computer-Initiated Interfaces**

The following computer – initiated interfaces were used:

- The menu system for the user is presented with a list of alternatives and the user chooses one; of alternatives.

- Questions – answer type dialog system where the computer asks question and takes action based on the basis of the users reply.

Right from the start the system is going to be menu driven, the opening menu displays the available options. Choosing one option gives another popup menu with more options. In this way every option leads the users to data entry form where the user can key in the data.

### **Error Message Design**

The design of error messages is an important part of the user interface design. As user is bound to commit some errors or other while designing a system the system should be designed to be helpful by providing the user with information regarding the error he/she has committed.

This application must be able to produce output at different modules for different inputs.

### **Performance Requirements**

Performance is measured in terms of the output provided by the application. Requirement specification plays an important part in the analysis of a system. Only when the requirement specifications are properly given, it is possible to design a system, which will fit into required environment. It rests largely in the part of the users of the existing system to give the requirement specifications because they are the people who finally use the system. This is because the requirements have to be known during the initial stages so that the system can be designed according to those requirements. It is very difficult to change the system once it has been designed and on the other hand designing a system, which does not cater to the requirements of the user, is of no use.

The requirement specification for any system can be broadly stated as given below:

- The system should be able to interface with the existing system
- The system should be accurate
- The system should be better than the existing system
- The existing system is completely dependent on the user to perform all the duties.

## CHAPTER 9

### SOURCE CODE

```
import warnings

warnings.filterwarnings('ignore')

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestRegressor

import xgboost as xgb

import lightgbm as lgb

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

import os

import joblib


df = pd.read_csv(r'Datasets/Dataset.csv')


df.info()


df.describe()


df.isnull().sum()
```

```
plt.figure(figsize=(10, 8))

sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt=".2f")

plt.title('Correlation Heatmap')

plt.show()
```

```
sns.pairplot(df, vars=df.columns, diag_kind='kde')

plt.show()
```

```
plt.figure(figsize=(12, 8))

for i, feature in enumerate(df.columns):

    plt.subplot(3, 5, i + 1)

    sns.histplot(df[feature], bins=20, kde=True)

    plt.title(feature)

plt.tight_layout()

plt.show()
```

```
x = df.drop(['PRICE'],axis = 1)

y = df['PRICE']
```

```
x_train, x_test, y_train, y_test = train_test_split(x,y, test_size = 0.2, random_state = 42)
```

```
def performance_metrics(algorithm, predict, testY):

    mse = mean_squared_error(testY, predict)
```

```
mae = mean_absolute_error(testY, predict)
```

```
r2 = r2_score(testY, predict)
```

```
print(algorithm + ' Mean Squared Error: {:.4f}'.format(mse))
```

```
print(algorithm + ' Mean Absolute Error: {:.4f}'.format(mae))
```

```
print(algorithm + ' R^2 Score: {:.4f}'.format(r2))
```

```
plt.scatter(testY, predict)
```

```
plt.plot([testY.min(), testY.max()], [testY.min(), testY.max()], '--r', lw=2)
```

```
plt.xlabel('True Values')
```

```
plt.ylabel('Predictions')
```

```
plt.title(algorithm + ' Predictions vs True Values')
```

```
plt.show()
```

```
rfr_model_path = 'model/RFR.joblib'
```

```
if os.path.exists(rfr_model_path):
```

```
    Regressor = joblib.load(rfr_model_path)
```

```
else:
```

```
    Regressor = RandomForestRegressor()
```

```
    Regressor.fit(x_train, y_train)
```

```
    joblib.dump(Regressor, rfr_model_path)
```

```
y_pred = Regressor.predict(x_test)
```

```
performance_metrics('Random Forest Regressor', y_pred, y_test)
```

```
xgb_model = 'model/xgb.npy'

if os.path.exists(xgb_model):

    model2 = np.load(xgb_model, allow_pickle = True).item()

else:

    model2 = xgb.XGBRegressor()

    model2.fit(x_train, y_train)

    np.save('model/xgb', model2)


y_pred1 = model2.predict(x_test)

performance_metrics('XGBoost Regression', y_pred1, y_test)


test=pd.read_csv("Datasets/test.csv")


pred = model2.predict(test)


test['pred'] = pred

test
```

## CHAPTER 10

### RESULTS AND DESCRIPTION

#### 10.1 Implementation Description

The implementation of the project code for predicting home prices using machine learning techniques can be broken down into several key steps, each representing a distinct flow in the overall process:

— **Data Loading and Preprocessing:**

- By importing necessary libraries and modules. It reads the dataset from a CSV file using Pandas' `read_csv()` function. Exploratory Data Analysis (EDA) techniques are applied to gain insights into the dataset, such as checking data types, summary statistics, and identifying missing values. Visualization techniques like heatmaps, pairplots, and histograms are used to understand data distributions and relationships between variables.

— **Data Splitting:**

- The dataset is split into features (independent variables) and the target variable (home prices). The train-test split is performed using scikit-learn's `train_test_split()` function, allocating a certain percentage (e.g., 80%) of the data for training and the remaining for testing.

— **Model Training:**

- Two regression models, `RandomForestRegressor` and `XGBoost`, are trained using the training data. For `RandomForestRegressor`, the code checks if a pre-trained model exists; if not, it initializes and fits a new model to the training data. Similarly, for `XGBoost`, the code either loads a pre-trained model or trains a new one. The models are trained to learn the relationships between the input features and home prices in the training data.

— **Model Evaluation:**

- A function called `performance_metrics()` is defined to evaluate the performance of each regression model. This function computes metrics such



as Mean Squared Error (MSE), Mean Absolute Error (MAE), and R-squared ( $R^2$ ) score to assess the model's accuracy. Additionally, it visualizes the predicted vs. true values using a scatter plot to visually inspect the model's performance.

— **Model Prediction:**

- Once trained, the models are used to make predictions on the test data. The predictions are compared against the actual home prices in the test data to assess the models' generalization performance.

— **Prediction on New Data:**

- Lastly, the code reads a separate testing dataset (new data) from a CSV file. The trained XGBoost model is used to predict home prices on this new dataset. The predictions are appended to the testing dataset for further analysis or deployment.

## 10.2 Dataset Description

The dataset captures various aspects of housing and neighborhood characteristics. These features are used to understand housing markets, predict prices, and make informed decisions.

— **CRIM (Crime Rate):**

- Represents the crime rate per capita by town.
- Indicates how many crimes occur relative to the population size.
- Higher values suggest areas with more crime incidents.

— **ZN (Residential Land Zoning):**

- Proportion of residential land zoned for lots over 25,000 sq.ft.
- Zoning regulations impact property development and land use.
- Higher values imply more spacious residential zones.

— **INDUS (Business Proportion):**

- Proportion of non-retail business acres per town.
- Reflects the industrial composition of the area.
- Higher values indicate more industrialized regions.

— **CHAS (Charles River Dummy):**

- Binary variable (1 if tract bounds river; 0 otherwise).
- Relevant for properties near the Charles River.
- 1 implies proximity to the river, affecting property value.

— **NOX (Nitric Oxides Concentration):**

- Measures nitric oxides concentration (parts per 10 million).
- Air pollution indicator.
- Higher values may impact health and property desirability.

— **RM (Average Rooms per Dwelling):**

- Indicates the average number of rooms in dwellings.
- Affects housing space and comfort.
- Higher values suggest larger homes.

— **AGE (Proportion of Older Units):**

- Proportion of owner-occupied units built before 1940.
- Reflects the age of housing stock.
- Older units may require more maintenance.

— **DIS (Distance to Employment Centers):**

- Weighted distances to five Boston employment centers.
- Proximity to jobs impacts property value.
- Lower values imply closer access to employment.

— **RAD (Accessibility to Highways):**

- Index of accessibility to radial highways.
- Relevant for commuting and transportation.
- Higher values indicate better highway access.

— **TAX (Property Tax Rate):**

- Full-value property-tax rate per \$10,000.
- Influences homeowners' expenses.
- Higher tax rates affect affordability.

— **PTRATIO (Pupil-Teacher Ratio):**

- Reflects the pupil-teacher ratio by town.
- Impacts education quality.
- Lower values suggest better student-teacher attention.

— **B (Proportion of Blacks):**

- Calculated as  $(1000(Bk - 0.63)^2)$ .
- Represents the proportion of Black residents.
- Social and demographic factor.

— **LSTAT (Lower Status Population %):**

- Percentage of lower-status population.
- Socioeconomic indicator.
- Higher values may correlate with lower property values.

— **PRICE (House Prices):**

- Likely the target variable.
- Represents house prices.
- Influenced by all other features.

### 10.3 Results Description

Boston House pricing dataset, showcasing various features such as crime rate, zoning, proximity to employment centers, and more, along with the corresponding house prices.

The figure 2 correlation heatmap illustrates the pairwise correlation between different features in the dataset. It provides insights into the relationships between variables, helping to identify potential predictors of house prices. The figure 3 pairplot visualizes pairwise relationships between different variables in the dataset. Each scatterplot represents the relationship between two variables, while the diagonal shows the distribution of each variable. The figure 4 histogram plot presents the distribution of each feature in the dataset. It helps in understanding the spread and frequency of values for individual variables. The figure 5 has performance metrics of a Random Forest Regressor (RFR) model are displayed in this figure, showing metrics such as Mean Squared Error (MSE), Mean Absolute Error (MAE), and R-squared ( $R^2$ ) score. The figure 6 prediction graph of the Random Forest Regressor (RFR) model illustrates the relationship between the true values and predicted values of house prices. It helps in visually assessing the accuracy of the model's predictions. The figure 7 presents the performance metrics of an XGBoost model, including MSE, MAE, and  $R^2$  score. The figure 8 prediction graph of the XGBoost model showcases the predicted values plotted against the true values of house prices. It provides a visual representation of the model's predictive capabilities. The figure 9 represents the process of uploading a test dataset for evaluation. It indicates the step where new data is imported for prediction using the trained machine learning models. The figure 10 proposed XGBoost model's predictions on the uploaded test dataset are displayed in this figure. It demonstrates how the model performs when making predictions on unseen data.

|     | CRIM    | ZN     | INDUS | CHAS  | NOX   | RM    | AGE  | DIS    | RAD | TAX   | \ |
|-----|---------|--------|-------|-------|-------|-------|------|--------|-----|-------|---|
| 0   | 0.00632 | 18.0   | 2.31  | 0.0   | 0.538 | 6.575 | 65.2 | 4.0900 | 1.0 | 296.0 |   |
| 1   | 0.02731 | 0.0    | 7.07  | 0.0   | 0.469 | 6.421 | 78.9 | 4.9671 | 2.0 | 242.0 |   |
| 2   | 0.02729 | 0.0    | 7.07  | 0.0   | 0.469 | 7.185 | 61.1 | 4.9671 | 2.0 | 242.0 |   |
| 3   | 0.03237 | 0.0    | 2.18  | 0.0   | 0.458 | 6.998 | 45.8 | 6.0622 | 3.0 | 222.0 |   |
| 4   | 0.06905 | 0.0    | 2.18  | 0.0   | 0.458 | 7.147 | 54.2 | 6.0622 | 3.0 | 222.0 |   |
| ..  | ...     | ...    | ...   | ...   | ...   | ...   | ...  | ...    | ... | ...   |   |
| 501 | 0.06263 | 0.0    | 11.93 | 0.0   | 0.573 | 6.593 | 69.1 | 2.4786 | 1.0 | 273.0 |   |
| 502 | 0.04527 | 0.0    | 11.93 | 0.0   | 0.573 | 6.120 | 76.7 | 2.2875 | 1.0 | 273.0 |   |
| 503 | 0.06076 | 0.0    | 11.93 | 0.0   | 0.573 | 6.976 | 91.0 | 2.1675 | 1.0 | 273.0 |   |
| 504 | 0.10959 | 0.0    | 11.93 | 0.0   | 0.573 | 6.794 | 89.3 | 2.3889 | 1.0 | 273.0 |   |
| 505 | 0.04741 | 0.0    | 11.93 | 0.0   | 0.573 | 6.030 | 80.8 | 2.5050 | 1.0 | 273.0 |   |
|     | PTRATIO | B      | LSTAT | PRICE |       |       |      |        |     |       |   |
| 0   | 15.3    | 396.90 | 4.98  | 24.0  |       |       |      |        |     |       |   |
| 1   | 17.8    | 396.90 | 9.14  | 21.6  |       |       |      |        |     |       |   |
| 2   | 17.8    | 392.83 | 4.03  | 34.7  |       |       |      |        |     |       |   |
| 3   | 18.7    | 394.63 | 2.94  | 33.4  |       |       |      |        |     |       |   |
| 4   | 18.7    | 396.90 | 5.33  | 36.2  |       |       |      |        |     |       |   |
| ..  | ...     | ...    | ...   | ...   |       |       |      |        |     |       |   |
| 501 | 21.0    | 391.99 | 9.67  | 22.4  |       |       |      |        |     |       |   |
| 502 | 21.0    | 396.90 | 9.08  | 20.6  |       |       |      |        |     |       |   |
| 503 | 21.0    | 396.90 | 5.64  | 23.9  |       |       |      |        |     |       |   |
| 504 | 21.0    | 393.45 | 6.48  | 22.0  |       |       |      |        |     |       |   |
| 505 | 21.0    | 396.90 | 7.88  | 11.9  |       |       |      |        |     |       |   |

[506 rows x 14 columns]

Figure 1: Presents the Sample dataset of the Boston House pricing dataset.

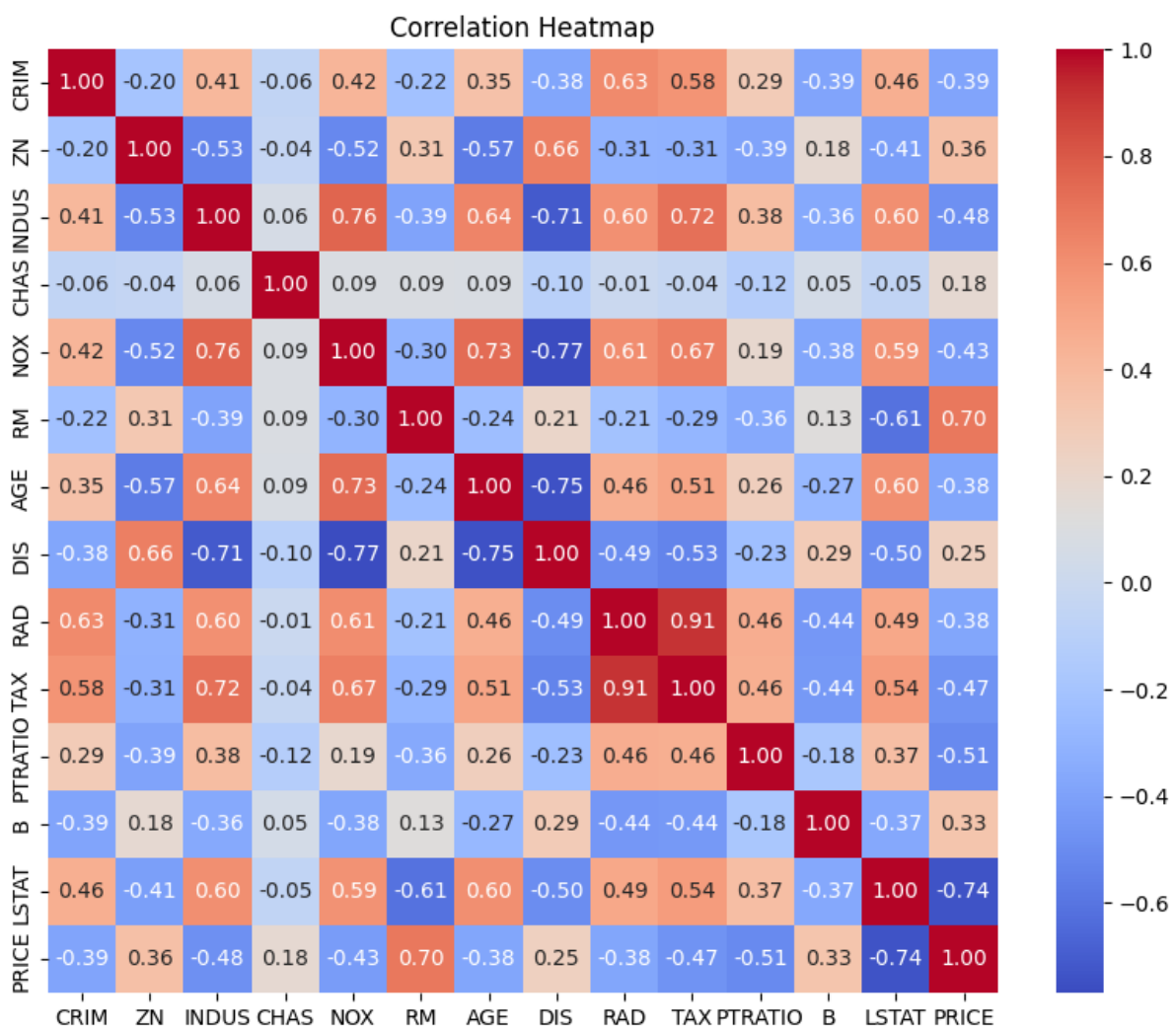


Fig. 2: Presents the correletion of minist dataset.

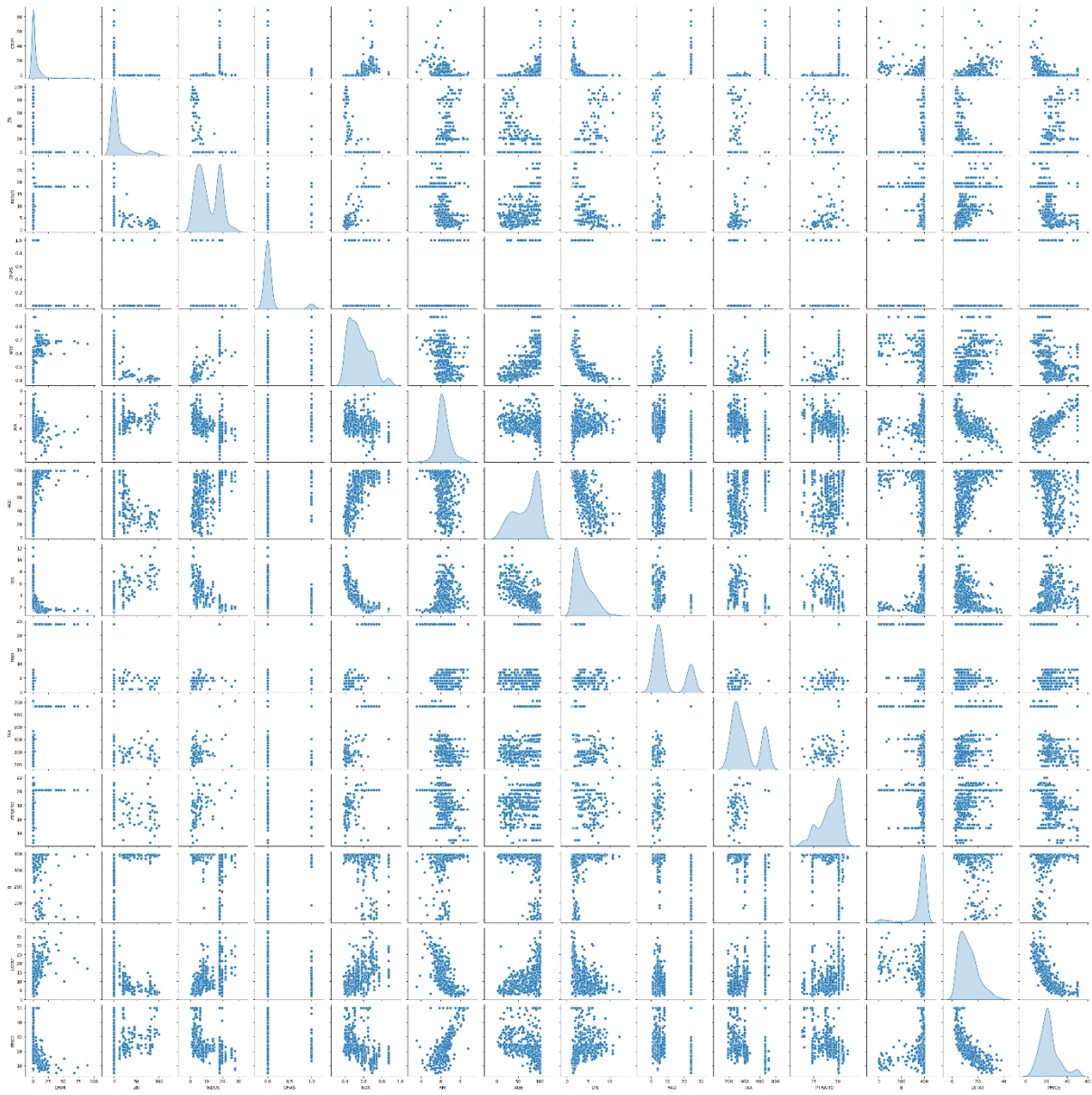


Fig. 3: Displays the pairplot of the dataset.

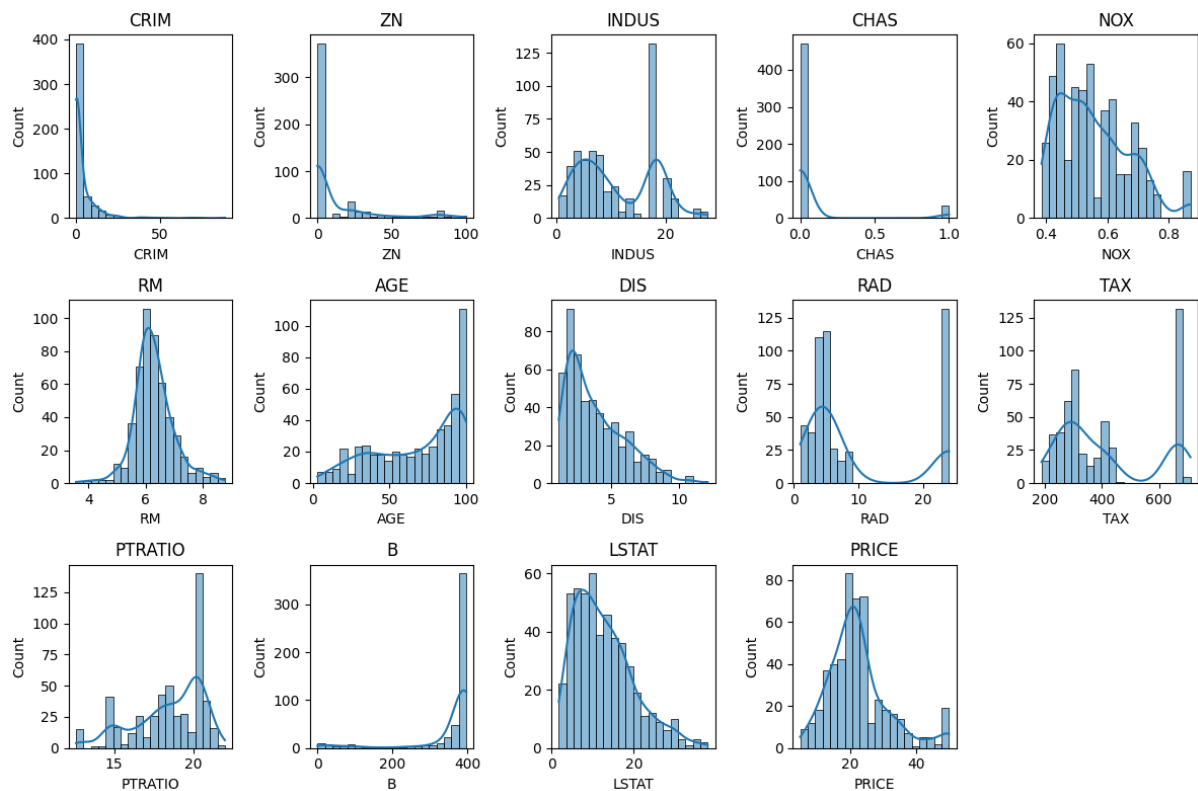


Fig. 4: Presents the histplot of dataset.

Random Forest Regressor Mean Squared Error: 9.1833  
 Random Forest Regressor Mean Absolute Error: 2.1365  
 Random Forest Regressor R<sup>2</sup> Score: 0.8748

Fig 5: Shows a performance metrics of of a RFR model.

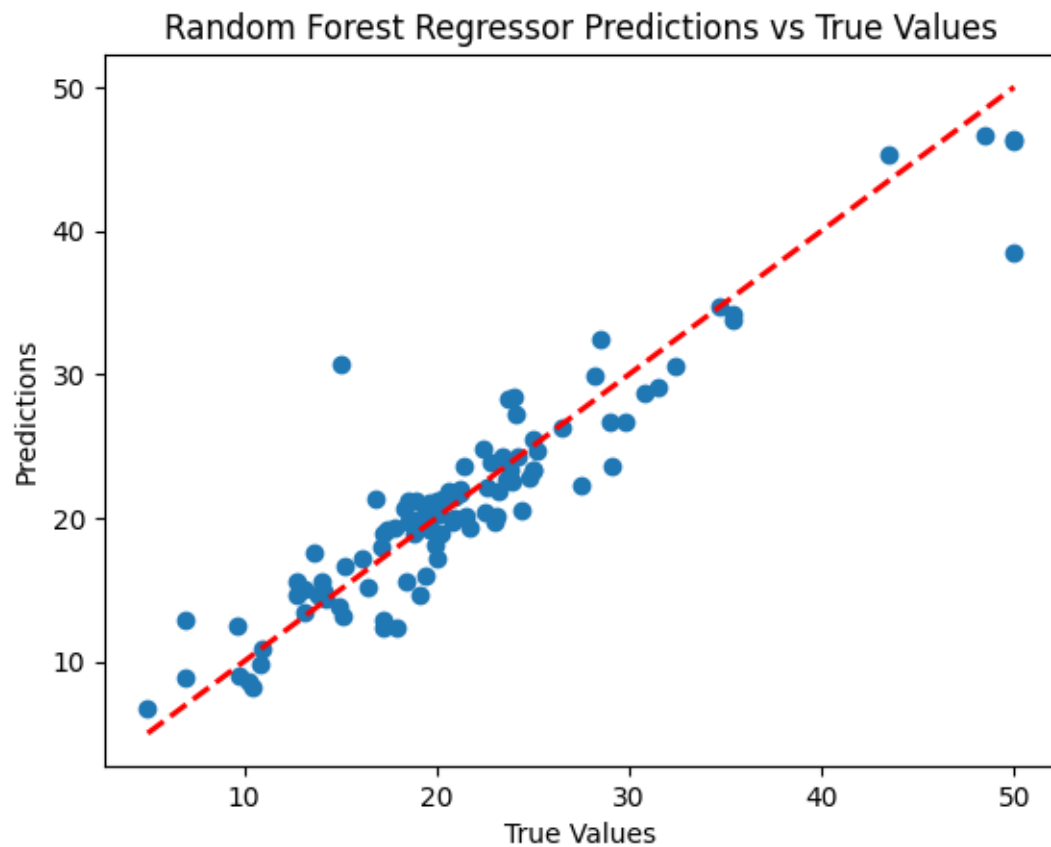


Fig 6: Prediction graph of RFR model.

XGBoost Regression Mean Squared Error: 6.5605  
XGBoost Regression Mean Absolute Error: 1.9575  
XGBoost Regression R<sup>2</sup> Score: 0.9105

Fig 7: Shows a performance metrics of of a XGBoost model.



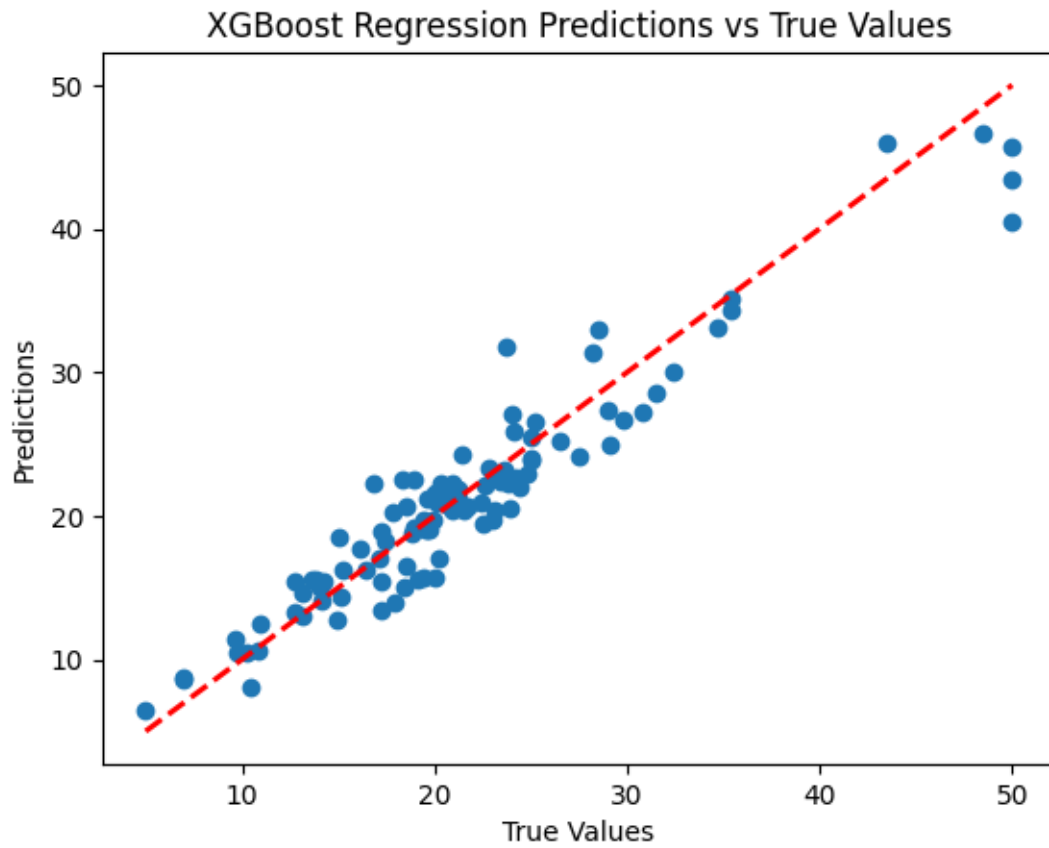


Fig 8: Prediction graph of XGBoost model.

|   | CRIM     | ZN   | INDUS | CHAS | NOX   | RM    | AGE  | DIS    | RAD  | TAX   | PTRATIO | B      | LSTAT |
|---|----------|------|-------|------|-------|-------|------|--------|------|-------|---------|--------|-------|
| 0 | 0.03738  | 0.0  | 5.19  | 0.0  | 0.515 | 6.310 | 38.5 | 6.4584 | 5.0  | 224.0 | 20.2    | 389.40 | 6.75  |
| 1 | 13.35980 | 0.0  | 18.10 | 0.0  | 0.693 | 5.887 | 94.7 | 1.7821 | 24.0 | 666.0 | 20.2    | 396.90 | 16.35 |
| 2 | 41.52920 | 0.0  | 18.10 | 0.0  | 0.693 | 5.531 | 85.4 | 1.6074 | 24.0 | 666.0 | 20.2    | 329.46 | 27.38 |
| 3 | 0.52058  | 0.0  | 6.20  | 1.0  | 0.507 | 6.631 | 76.5 | 4.1480 | 8.0  | 307.0 | 17.4    | 388.45 | 9.54  |
| 4 | 0.04932  | 33.0 | 2.18  | 0.0  | 0.472 | 6.849 | 70.3 | 3.1827 | 7.0  | 222.0 | 18.4    | 396.90 | 7.53  |

Fig 9: Presents the uploading of test dataset.

|   | CRIM     | ZN   | INDUS | CHAS | NOX   | RM    | AGE  | DIS    | RAD  | TAX   | PTRATIO | B      | LSTAT | pred      |
|---|----------|------|-------|------|-------|-------|------|--------|------|-------|---------|--------|-------|-----------|
| 0 | 0.03738  | 0.0  | 5.19  | 0.0  | 0.515 | 6.310 | 38.5 | 6.4584 | 5.0  | 224.0 | 20.2    | 389.40 | 6.75  | 20.694065 |
| 1 | 13.35980 | 0.0  | 18.10 | 0.0  | 0.693 | 5.887 | 94.7 | 1.7821 | 24.0 | 666.0 | 20.2    | 396.90 | 16.35 | 12.686036 |
| 2 | 41.52920 | 0.0  | 18.10 | 0.0  | 0.693 | 5.531 | 85.4 | 1.6074 | 24.0 | 666.0 | 20.2    | 329.46 | 27.38 | 8.495610  |
| 3 | 0.52058  | 0.0  | 6.20  | 1.0  | 0.507 | 6.631 | 76.5 | 4.1480 | 8.0  | 307.0 | 17.4    | 388.45 | 9.54  | 25.139647 |
| 4 | 0.04932  | 33.0 | 2.18  | 0.0  | 0.472 | 6.849 | 70.3 | 3.1827 | 7.0  | 222.0 | 18.4    | 396.90 | 7.53  | 31.328337 |

Fig 10: Proposed XGBoost model Prediction on uploaded test dataset.

# **CHAPTER 11**

## **CONCLUSION AND FUTURE SCOPE**

In conclusion, research in predicting home prices using regression analysis techniques has made significant strides in recent years, offering valuable insights and opportunities for stakeholders in the real estate industry. By leveraging big data and machine learning technologies, researchers have developed predictive models capable of accurately forecasting home prices and uncovering valuable insights into market dynamics.

Looking ahead, the future scope of research in this area is promising, with opportunities for further advancements in predictive modeling techniques, data analytics, and computational methods. Future research endeavors may focus on refining existing models, incorporating additional data sources, and exploring novel algorithms to improve prediction accuracy and scalability. Additionally, there is potential for research in predictive modeling to extend beyond home prices and encompass other aspects of the real estate market, such as rental prices, property demand forecasting, and real estate investment analysis.

## REFERENCES

- [1] Mansi Bosamia, 'Positive and Negative Impacts of Information and Communication Technology in our Everyday Life', International Conference on "Disiplinary and Interdisciplinary Approaches to Knowledge Creation in Higher Education: CANADA and INDIA (GENESIS 2013).
- [2] Rahul Reddy Nadikattu, 'THE EMERGING ROLE OF ARTIFICIAL INTELLIGENCE IN THE MODERN SOCIETY', International Journal Of Creative Research Thoughts, 2016.
- [3] Ravi Manne and Sneha C Kantheti, 'Application of Artificial Intelligence in Healthcare: Changes and Challenges', Current Journal 5 of Applied Science and Technology, 2021, DOI:10.9734/CJAST/2021/v40i631320
- [4] Woubishet Zewdu Taffese, 'A Survey on Application of Artificial Intelligence in Real Estate Industry', 3rd International Conference on Artificial Intelligence in Engineering and Technology, 2006.
- [5] Ferreira, F. G. D. C., Gandomi, A. H., & Cardoso, R. T. N. 'Artificial Intelligence Applied to Stock Market Trading: A Review', IEEE, 2021, doi:10.1109/access.2021.3058133.
- [6] Anandharajan, T. R. V., Hariharan, G. A., Vignajeth, K. K., Jijendiran, R., & Kushmita. (2016). 'Weather Monitoring Using Artificial Intelligence'. 2016 2nd International Conference on Computational Intelligence and Networks (CINE). doi:10.1109/cine.2016.26.
- [7] Tong, W., Hussain, A., Bo, W. X., & Maharjan, S., 'Artificial Intelligence for Vehicle-to-Everything: a Survey', IEEE, 2019 ,doi:10.1109/access.2019.2891073.
- [8] Sumit Das, Aritra dey, Akash Paul and NAbamita Roy, 'Applications of Artificial Intelligence in Machine Learning: Review and Prospects', International Journal of Computer Applications, 2015, DOI:10.5120/20182-2402.
- [9] Avneet Pannu, 'Artificial Intelligence and its Application in Different Areas', International Journal of Engineering and Innovative Technology (IJEIT), Volume 4, Issue 10, April 2015.
- [10] John A. Bullinaria, 'IAI : The Roots, Goals and Sub-fields of AI', 2005. <http://www.cs.bham.ac.uk/~jxb/IAI/w2.pdf>

- [11] Bahia, I. S. (2013). A Data Mining Model by Using ANN for Predicting Real Estate Market: Comparative Study. *International Journal of Intelligence Science*, 03(04), 162- 169. doi:10.4236/ijis.2013.34017.
- [12] Mu, J., Wu, F., & Zhang, A. (2014). Housing Value Forecasting Based on Machine Learning Methods. *Abstract and Applied Analysis*, Volume 2014 (2014), Article ID 648047, 7 pages. Retrieved April 2017, from <https://www.hindawi.com/journals/aaa/2014/648047/>.
- [13] Eli Beracha, Ben T Gilbert, Tyler Kjorstad, Kiplan womack, "On the Relation between Local Amenities and House Price Dynamics", *Journal of Real estate Economics*, Aug. 2016.
- [14] Stephen Law, "Defining Street-based Local Area and measuring its effect on house price using a hedonic price approach: The case study of Metropolitan London", *Cities*, vol. 60, Part A, pp. 166–179, Feb. 2017.
- [15] Binbin Lu, Martin Charlton, Paul Harris & A. Stewart Fotheringham, "Geographically weighted regression with a non-Euclidean distance metric: a case study using hedonic house price data", *International Journal of Geographical Information Science*, pp. 660-681, Jan 2014.
- [16] Marco Helbich, Wolfgang Brunauer, Eric Vaz, Peter Nijkamp, "Spatial Heterogeneity in Hedonic House Price Models: The Case of Austria", *Urban Studies*, vol. 51, Issue 2, Feb. 2014.
- [17] Sean Holly, M. Hashem Pesarana, Takashi Yamagata, "A spatio-temporal model of house prices in the USA", *Journal of Econometrics*, vol. 158, Issue 1, pp. 160–173, Sep. 2010.
- [18] Joep Steegmans, Wolter Hassink, "Financial position and house price determination: An empirical study of income and wealth effects", *Journal of Housing Economics*, vol. 36, pp. 8-24, June 2017.